Algorithms for Data Science
Cheatsheet for Exercises 1

Loop invariants: To prove that an algorithm is correct, one can often use a *loop invariant*. Formulating an invariant also helps to find mistakes!
An invariant should be

- true, and
- helpful.

We look now at these two points in detail.

A loop invariant is defined with respect to a loop in the program. It is any logical statement on how the variables are related to each other. The statement must be true for every iteration of the loop (therefore the name "invariant"—something that does not change). Normally, the invariant concerns the moment of the iteration, when the condition in the loop head is checked.

As an example, consider the following algorithm.

```
i = 0
j = 0
k = 0
n = 100
while i ≠ n do
    i = i + 1
    j = j + 2
    k = k + 3
```

A possible invariant is "We have $2i = j$."

To prove that an invariant is always true, it suffices

1. to show that it is true in the beginning of the first loop iteration, and
2. to show that, if it is true at the beginning of a loop iteration, it is also true at the end of that iteration (which implies that it is true at the beginning of the next iteration).

In our example, we have $2i = 0 = j$ in the beginning of the first while loop iteration. Thus, the first step holds. Regarding the second step of our proof, suppose that the invariant is true for some value $i = i'$ and $j = j'$, that is, $2i' = j'$ by our assumption. During the loop iteration, we increase i by 1 and j by 2. Thus, at the end of that iteration, we have $i = i' + 1$ and $j = j' + 2$. Hence, if we check the invariant statement, we see that $2i = 2(i' + 1) = 2i' + 2 = j' + 2$ (recall that $2i' = j'$). This means that the invariant holds also at the end of that statement. Thus, it is true in the beginning of every iteration.

So, are invariants always helpful? The answer is: no, sometimes they might not be helpful! For instance, if we wanted to prove that $k = 3n$ at the end of our algorithm above, our invariant will be of no help as it makes no statements about k. However, if we wanted to prove that $j = 2n$, our invariant is perfect: since our invariant is always true, it is also true at the moment when we leave the while loop. On the other hand, we know that the while loop is only left if the loop condition $i \neq n$ is violated, that is, when $i = n$. Thus, at the end, we have $j = 2i = 2n$.
Note that for the same algorithm, there are infinitely many correct invariant statements. But we are interested only in one that helps us to show that our algorithm is correct. Also note that an invariant alone is not enough to prove the correctness of an algorithm. We also need to show that all the loops in the algorithm stop at some point, that is, that the algorithm terminates. In our example, we could argue that the difference between n and i drops by exactly 1 in each iteration, and therefore will be 0 after a finite number of iterations.

Induction: For completeness, let us note that invariants are strongly related to the concept of "proof by induction". In fact, proofs by invariants are just a special case. In a proof by induction, we want to show that a logical statement depending on some variable i is true for all (infinitely many) values of i (from some given range). The statement can of course depend on more than one variable.
As an example, consider the statement: "We have $2^0 + \cdots + 2^i = 2^{i+1} - 1$." As the range for i, let us choose $i \geq 0$. To prove the correctness,

1. we show that the statement is true for the smallest i (in our example: $i = 0$ yields $2^0 = 1$ and $2^{0+1} - 1 = 2 - 1 = 1$), and

2. by using the assumption that it is true for some i, we show that the statement is also true for $i+1$. In our example: $2^0 + \cdots + 2^{i+1} = (2^0 + \cdots + 2^i) + 2^{i+1} = (2^{i+1} - 1) + 2^{i+1} = 2 \cdot 2^{i+1} - 1 = 2^{i+2} - 1$.

This is enough. These two steps automatically imply that our statement is true for all infinite many i. Why? The second step can be thought of as writing an algorithm whose input is i and whose output is a proof for the correctness of the statement for $i+1$ where the proof assumes that the statement is true for i. By taking the proof for the smallest value for i, say $i = 0$, (step 1) and calling our algorithm with this value (step 2), we obtain in total a proof for the value $i + 1 = 1$. By repeatedly calling our algorithm for $i = 1$, $i = 2$, ..., we get in total a proof that our statement is true for any possible value for i.

$$\text{proof}(i = 0) \rightarrow \text{proof}(i = 1) \rightarrow \text{proof}(i = 2) \rightarrow \ldots$$

In our course:

- `x++` in Pseudocode (and in C++) means $x = x + 1$ (increasing the variable x by 1).
- `x--` means $x = x - 1$.
- `for i=1 to n` in Pseudocode means the same as
  - `for (int i=1; i <= n; i++)` in C++, and
  - `for i in range(1,n+1)` in Python.

  Importantly, the last iteration is for $i = n$.
- `a[2..n]` in Pseudocode defines an array a with available index positions from 2 to n (thus, this array contains $n-2+1 = n-1$ elements; hence, the *size* of a is $n-1$). An array is the same as a Python list or a C++ vector. Note that in Pseudocode, we can specify the range of available positions of an array, whereas in Python and C++ the first position is always 0 and only the last position is specified.
- To create a new array in Pseudocode, you can write any of this:
  1. `a[2..n] = new array`,
  2. `a[2..n] new array`, or simply just
  3. `a[2..n]`

  If the array elements are assumed to have an initial value c just write "filled with c". For example, `a[2..n] new array filled with 0s`. If you want to specify the type of the elements, for example that they should be integers, then write `a[2..n] new array of integers`. (If not specified, we assume an array to be of type integers.)
- `b = a[i..j]` creates a new array b that is a copy of the subarray $a[i..j]$. For example, if the array a is defined as `a[1..n]`, then `b = a[1..n/2]` is a new array that contains the first $n/2$ elements of a (as copies).
- We assume that variables created within a function call are **automatically deleted** at the end of that function call.
- `//` and `/*` (C++ style) and `#` (Python style) begin a comment. Thus, when writing `i = 2 // + 3`, we set the value of i to 2 and not to 5 as "+3" is just a comment.
- If b is a Boolean value (true/false), then `if b` is short for `if b==`**`True`** and `if !b` is short for `if b==`**`False`**.

Algorithms for Data Science
Exercise 1
(with Solutions)

See the cheatsheet for valuable remarks!

Task 1.1: *Coin Problem*

You are given eight equally looking coins and a pair of scales. One of the coins is forged and has smaller weight, whereas all the other coins weigh the same. A single weighing on the scales consists of comparing the weight of any two (disjoint) subsets of the coins. Note that the weighting instrument does not show the actual weight.

(a) Can you find the forged coin by using the scales three times?

Solution: Yes, e.g., by binary search.

(b) Is it possible to find it using the scales only two times?

Solution: Yes, first compare two three-element sets. Then one weighing more is enough to determine the forged coin.

(c) If we are allowed to do k weighings, what is the maximum number of coins among which we can find the forged coin?

Solution: 3^k.

To come up with the formula, first find the exact maximum number of coins for small values of k ($k = 0, 1, 2$). To do so, you have to (1) give a strategy for finding the forged coin and to (2) argue that if we have more than 3^k coins, then it is impossible to find the forged one using the scales only k times.

k	maximum number of coins
0	1
1	3
2	9

(For $k = 0$ we use that there is exactly one forged coin.)

The task did not ask for a proof of the formula (since such a proof is a little out of the scope of our course), but for completeness we sketch the proof idea:

The strategy for general k is a generalization of the previous subtask: partition the 3^k coins into three sets of equal size 3^{k-1} and use the scales on any two of them. As the outcome, we know which of the three sets contains the forged coin. Proceed recursively on this set. After k steps, we arrive at a set of size $3^{k-k} = 3^0 = 1$ and we are done.

Next we show that it is impossible to find the forged coin among more than 3^k coins by using the scales only k times. Our proof uses the induction technique. Suppose we have shown

that for more than 3^{k-1} coins, we need at least k weighings. (Indeed, we have already shown this claim for the smallest value(s) of k; see the table above.) Consider a set with more than 3^k coins. A weighing is nothing more than a partition of the coins into three sets and the information to which set the forged coin belongs to. At least one of the three sets contains more than $3^k/3 = 3^{k-1}$ coins. In the worst case, it is the set containing the forged coin. By assumption, we need at least k more weighings to find the forged coin from this set. Thus, in the worst case, we need in total at least $k+1$ weighings if we have more than 3^k coins.

Task 1.2: *Divide by Two*

In this task, we want to divide a given integer by two without using the division operation. Consider the following algorithm.

Algorithm 1:

Input: integer $n \geq 0$
Output: the value $n/2$ (possibly fractional)
$\ell = 0$
$r = n$
while $\ell \neq r$ **do**
    $\ell = \ell + 1$
    $r = r - 1$
return ℓ

(a) How often does the while loop repeat in dependence of n?

Solution: For even n: $n/2$ times.
For odd n: infinite loop!

(b) The program is not correct. What is the error? Fix the mistake!

Solution:
For odd n, the program does not terminate as ℓ and r "jump over each other."
There are (at least) two different ways to fix the error:

1. Change $\ell = \ell + 1$ to $\ell = \ell + 0.5$ and $r = r - 1$ to $r = r - 0.5$.

Algorithm 2:

Input: integer $n \geq 0$
Output: the value $n/2$ (possibly fractional)
$\ell = 0$
$r = n$
while $\ell \neq r$ **do**
    $\ell = \ell + 0.5$
    $r = r - 0.5$
return ℓ

2. Change the loop condition to $\ell < r$. After the loop, depending on whether n is even or not, return ℓ or $r + 0.5$. (Note that if n is even, then $\ell = r$, otherwise $r + 1 = \ell$.)

Algorithm 3:

Input: integer $n \geq 0$
Output: the value $n/2$ (possibly fractional)
$\ell = 0$
$r = n$
while $\ell < r$ **do**
    $\ell = \ell + 1$
    $r = r - 1$
if $\ell = r$ **then**
    return ℓ
else // $r + 1 = \ell$
    return $r + 0.5$

(c) Prove the correctness of the fixed algorithm with a loop invariant. Show also that the algorithm terminates.

Solution:

Proof of Algorithm 2 Our algorithm terminates: after each iteration of the loop, the difference $r - \ell$ gets smaller by 1. Intially the difference is the nonnegative integer n. Thus, after finitely many steps we have $r - \ell = 0$ which is equivalent to $\ell = r$ which means that the loop condition will be violated.

To prove correctness, we choose "$n = \ell + r$" to be our loop invariant. (Intuitively, the statement says that "$n/2$ is the middle point of the interval $[\ell, r]$.")

The invariant is true:

- In the beginning of the first iteration (that is, the first time when the loop condition is checked by the program), we have $\ell = 0$ and $r = n$, thus $\ell + r = n$ and the invariant statement holds.
- Assume that in the beginning of some interation (where the loop is not exited) the invariant statement holds for some values ℓ' and r'. At the end of that iteration we have $\ell = \ell' + 0.5$ and $r = r' - 0.5$. Thus $\ell + r = \ell' + 0.5 + r' - 0.5 = \ell' + r'$ which is equal to n by assumption; hence, the invariant statement holds also for the next iteration.

The output of Algorithm 2 is correct (hence, the invariant is helpful): At the end, we exit the loop only if $\ell = r$. Together with our invariant, we have $n = \ell + r = 2\ell$, that is, $n/2 = \ell$; our output is correct!

Proof of Algorithm 3 Our algorithm terminates: after each iteration of the loop, the difference $r - \ell$ gets smaller by 2; thus, after a finite number of iterations we have $r - \ell \leq 0$, that is, $\ell \geq r$ which means that the loop condition will be violated.

To prove correctness, we choose the following statement as our loop invariant: "(i) $n = \ell + r$, (ii) $\ell \leq r + 1$ and (iii) ℓ, r are integers". (Intuitively, fact (i) says that "$n/2$ is the middle point of the interval $[\min(\ell, r), \max(\ell, r)]$.")

The invariant is true:

- In the beginning of the first iteration (that is, the first time when the loop condition is checked by the program), we have $\ell = 0$ and $r = n$, thus (i) holds as $\ell + r = n$, and (ii) and (iii) hold, as $r = n$ is a nonnegative integer and $\ell = 0 \leq r \leq r+1$; hence, the invariant holds in the beginning.
- Assume that in the beginning of some interation (where the loop is not exited) the invariant statement holds for some values ℓ' and r'. At the end of that iteration, we have $\ell = \ell' + 1$ and $r = r' - 1$. Thus, ℓ and r are also integers (fact (iii) also holds) and given that the loop condition was satisifed with $\ell' < r'$, we have $\ell \leq r+1$ (fact (ii) holds). Fact (iii) also holds for ℓ and r as $\ell + r = \ell' + 1 + r' - 1 = \ell' + r'$ which is equal to n by assumption. Thus, the invariant holds also for the next iteration.

The output of Algorithm 3 is correct (hence, the invariant is helpful): At the end, we exit the loop when $\ell \not< r$, that is, when (iv) $\ell \geq r$. Assuming that our invariant is true, facts (ii), (iii) and (iv) imply that either $\ell = r$ or $r + 1 = \ell$. If $\ell = r$, then fact (i) implies $n = \ell + r = 2\ell$, that is, $n/2 = \ell$ and our output is correct for this case! In the other case of $r + 1 = \ell$, fact (i) implies $n = \ell + r = 2r + 1$, that is, $n/2 = r + 1/2$ and, again, our output is correct!

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 1.3: *Smallest Element*

In this task, we want to find a smallest element in an unordered array.

(a) Consider the following algorithm.

Algorithm 4: Algorithm A

```
Input: array a[1..n]
Output: the position of a smallest element
minPos = −1
for i = 1 to n do
    isSmallest = true
    for j = 1 to n do
        if a[i] ≥ a[j] then
            isSmallest = false
    if isSmallest then
        minPos = i
return minPos
```

i. What is the output of the algorithm? Fix the error and discuss why the algorithm is now correct!

> **Solution:** The output is always -1 as the line $minPos = i$ is never reached since for every i, when the inner loop reaches $j = i$, the condition $a[i] \geq a[j]$ is obviously satisfied and thus $isSmallest$ is set to false.
> To fix this problem, change $a[i] \geq a[j]$ to $a[i] > a[j]$. This is enough. Now $isSmallest$ is set to false only if there exists a smaller value than $a[i]$, that is, if $a[i]$ is not the smallest.

ii. How often does the (repaired) algorithm compare two elements of a (that is, $a[i] \geq a[j]$) in dependence of n in the worst case?

> **Solution:** n^2.

iii. Suppose that the running time depends only on comparing two elements of a. Further suppose that your computer can compare two elements of a in one millisecond. What is the worst case running time of the (fixed) algorithm for $n = 1000\ 000$ on your computer?

> **Solution:** About 32 years.

(b) Write a faster algorithm by filling the gaps of Algorithm B below.

Algorithm 5: Algorithm B

Input: array $a[1..n]$
Output: the position of a smallest element

```
1 minPos = 1
2 j = 2
3 while ..... do
4 |   .....
5 |   .....
6 |   j = j + 1
7 return minPos
```

Solution:

Algorithm 6: Algorithm B

Input: array $a[1..n]$
Output: the position of a smallest element

```
1 minPos = 1
2 j = 2
3 while j ≤ n do
4 |   if a[minPos] > a[j] then
5 |   |   minPos = j
6 |   j = j + 1
7 return minPos
```

i. How often does Algorithm B compare two elements of a

Solution: $n - 1$

ii. Suppose that the running time depends only on comparing two elements of a. Further suppose that your computer can compare two elements of a in one millisecond. What is the worst case running time of the algorithm for $n = 1000\ 000$ on your computer?

Solution: About 17 minutes.

iii. Formulate an invariant depending on a, j and $minPos$. Use the invariant to argue that Algorithm B is correct.

Solution: Invariant: "$minPos$ is the position of the minimum element of the subarray $a[1..j]$." It is true for $j = 1$ and, assuming trueness for j, also true for $j + 1$. Thus, the invariant is true for all values of j, in particular for $j = n$ when we leave the loop.

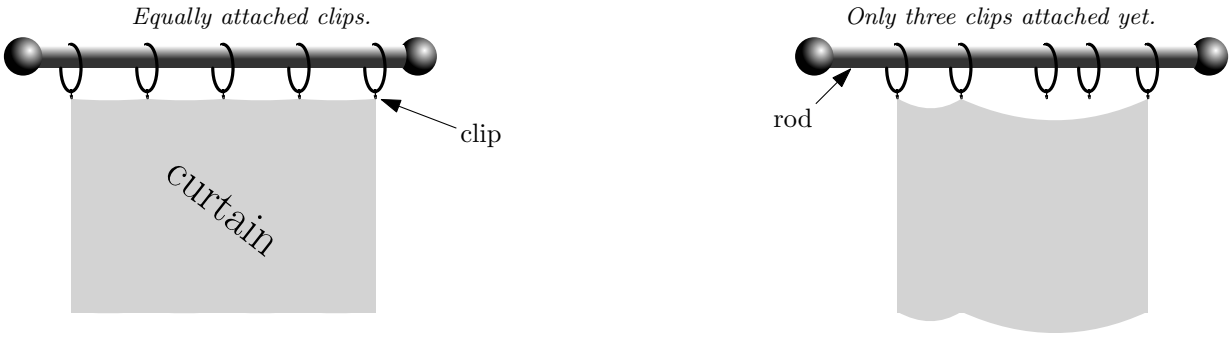
(c) Discuss why any algorithm needs at least $n-1$ comparisons to find a minimum element in the worst case.

Solution: In the beginning, every element is a possible candidate for the minimum element. After each comparison, the set of candidates decreases by at most one element.

Task 1.4: *Curtain Problem - a real-world problem brought by the lecturer's wife*

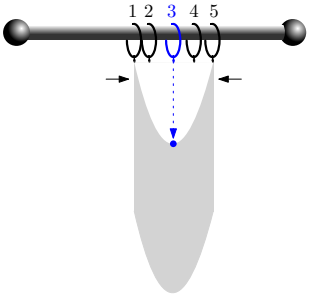
Suppose that you washed the curtain of your window and now you want to hang it up again. To do so, you have to attach the curtain to the five ring clips lying loosely on a rod above the window. It is important that the clips are equally spaced on the curtain, otherwise it won't look nice.

Unfortunately, you don't have any tool to measure the distances in order to find the right position for attaching the clips. However, you are a smart student and you want to use gravity and the idea underlying binary search to solve this problem.



(a) Describe how to find equally spaced positions for the clips. Note that you can move the ring clips along the rod and that the curtain is obeying the laws of gravity. Your first step is to attach the top corners of the curtain to the left- and right-most clip.

Solution: After the two top corners are attached to the left- and right-most clip rings, we slide the two clips to each other. By gravity, the curtain folds into half and we can determine the position to attach the middle clip; see the figure. We proceed recursively.



(b) Does your procedure also work for seven clips?

Solution: No, in the recursive step we have an even number of clips and therefore no unique middle clip.

(c) For what numbers of clips does your procedure work?

Solution: For any number where there is a unique middle clip in each recursive step. Thus, for $2 + (1 + 2 + \cdots + 2^{i-1}) = 2 + 2^i - 1 = 2^i + 1$ clips.

Task 1.5: *Guess my number!—Level 2*

Consider the variant of the game *Guess my number!* when there is no upper bound on the integer number x to be guessed. We only know $x \geq 8$. Propose an algorithm to solve the problem using at most $c \cdot \log_2 x$ questions where c can be any constant number that does not depend on x; e.g. $c = 8$.

Hint: First, try to find an upper bound r of x with as few questions as possible. Then, run Binary Search from the lecture. You can assume that it needs only $\log_2 n$ questions when n is the size of the given interval containing x.

Solution: We do exponential search to find r by testing $r = 2^0, 2^1, \ldots, 2^i$ (we could start from $r = 8 = 2^3$). When we found such an r, then we have $x \leq r < 2x$. Recall that we choose r such that $r = 2^i$ for some i. Thus, we needed at most $i+1$ questions for finding r. Since we have $i = \log_2 r$ and $\log_2 r \leq \log_2(2x)$, we can bound the number of questions for finding r by $i + 1 \leq \log_2(2x) + 1$. As the interval length (of $[3..r]$) is less than $2x$, we need at most $\log_2 2x$ questions to find x by our assumption. Thus, the total number of questions is bounded by $2\log_2(2x)+1 = 2\log_2 x+3 \leq 3\log_2 x$ (we use the rule $\log_2(2x) = 1 + \log_2 x$ and that $\log_2 x \geq 3$ as $x \geq 8$).

Task 1.6: *Real Time Running Times*

Given are five algorithms for the same (unknown) problem that get an array of size n as the input. The number of operations of the algorithms in dependence of n are $\log_2 n$, n, $n \log_2 n$, n^2 and 2^n, respectively. Give their running times using an adequate time unit (e.g. nanoseconds,..., years) in each case for different values of n. You can round the time values to integers and ignore running times above 10 years.

(a) Suppose your computer can do 10 operations per second.

	$\log_2 n$	n	$n \log_2 n$	n^2	2^n
n=10					
n=20					
n=21					
n=1000					
n=1000 000					
n=1000 000 000					

Solution:

	$\log_2 n$	n	$n \log_2 n$	n^2	2^n
n=10	300ms	1s	3s	10s	2min
n=20	400ms	2s	9s	40s	1d
n=21	400ms	2s	9s	44s	2d
n=1000	1s	2min	17min	1d	$> 3 \cdot 10^{292}$y
n=1000 000	2s	1d	23d	3k y	
n=1000 000 000	3s	3y	95y	3000 mio y	

(b) Suppose your computer can do 1000 000 operations per second.

	$\log_2 n$	n	$n \log_2 n$	n^2	2^n
n=10					
n=20					
n=21					
n=1000					
n=1000 000					
n=1000 000 000					

Solution:

	$\log_2 n$	n	$n \log_2 n$	n^2	2^n
n=10	3μs	10μs	33μs	100μs	1ms
n=20	4μs	20μs	86μs	400μs	1s
n=21	4μs	21μs	92μs	441μs	2s
n=1000	10μs	1ms	10ms	1s	$> 3 \cdot 10^{287}$y
n=1000 000	20μs	1s	20s	12d	
n=1000 000 000	30μs	17min	8h	32k y	

Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 1.7: *Computing the Square Root*

Write an algorithm that, given a nonnegative integer n, computes the value $\lfloor \sqrt{n} \rfloor$. The algorithm is not allowed to call other functions and is allowed to have only a single loop. This loop is not allowed to iterate more than $\lceil \log_2 n \rceil + 1$ times. Prove the correctnes of your algorithm.

Solution: Do binary search in the interval $[1, n]$ to find an integer m such that $m \cdot m \leq n < (m+1) \cdot (m+1)$. Use appropriate loop invariants to show that your algorithm terminates and that the output is correct.

Algorithms for Data Science
Cheatsheet for Exercises 2

Recall the definition of **asymptotic notation**:

- $f(n) = O(g(n))$ if there exist constants $C > 0$ and $n_0 \geq 0$ such that, for all $n \geq n_0$, $f(n) \leq C \cdot g(n)$.
- $f(n) = \Omega(g(n))$ if there exist constants $C > 0$ and $n_0 \geq 0$ such that, for all $n \geq n_0$, $f(n) \geq C \cdot g(n)$.
- $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

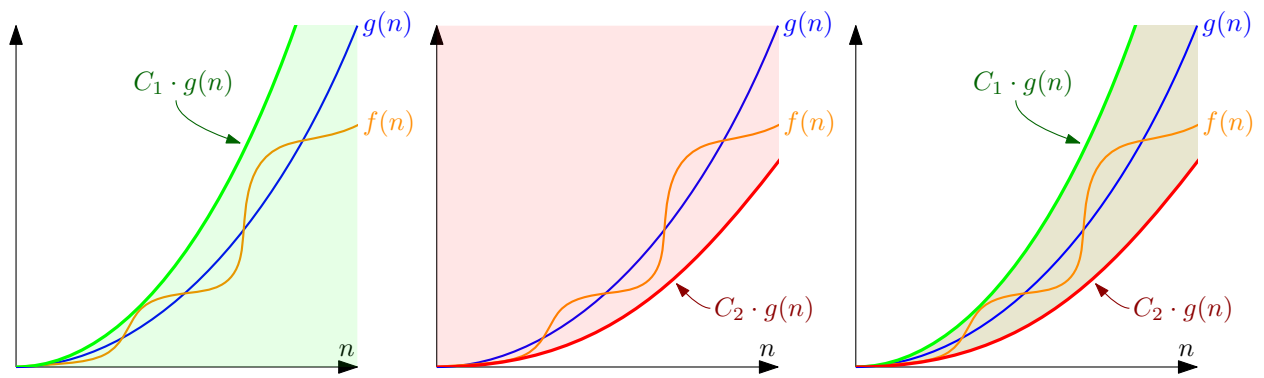
Equivalently, $f(n) = \Omega(g(n))$ if $g(n) = O(f(n))$—see Task 2.3(b)

Intuitively:

- $f(n) = O(g(n))$ if g is an **"upper bound"** on f.
- $f(n) = \Omega(g(n))$ if g is a **"lower bound"** on f.
- $f(n) = \Theta(g(n))$ if f and g are asymptotically **equal**.

Furthermore: f is asymptotically **faster** than g if f is not a "lower bound" on g, that is, if $g(n) \neq \Omega(f(n))$.

Graphically:



(a) $f(n) = O(g(n))$. (b) $f(n) = \Omega(g(n))$. (c) $f(n) = \Theta(g(n))$.

Explanation:

(a) *In this example, there is a fixed number C_1 such that $f(n)$ lies below the function $C_1 \cdot g(n)$ for all $n \geq 0$ (that is, we can set $n_0 = 0$). Thus, we have $f(n) = O(g(n))$.*

(b) *Similarly, $f(n) = \Omega(g(n))$ as there is a fixed number C_2 such that $f(n)$ lies above $C_2 \cdot g(n)$ for all $n \geq 0$.*

(c) *Since $f(n)$ lies in the "corridor" defined by C_1, C_2 and $g(n)$, we have $f(n) = \Theta(g(n))$.*

Ensure you understand this: ***The faster an algorithm is, the slower its running time is growing!***
(Since a faster algorithm takes less time, it's running time is growing more slowly.)

In our course:

- log always means $\log_2$, and
- *running time* always means the *worst-case* running time (if not otherwise noted).

Recall that $\log_b a$ is defined as the x satisfying the equality $b^x = a$.

Def.: A **simplification** of $f(n)$ is a $g(n)$ that looks as "simple" as possible and that satisfies $f(n) = \Theta(g(n))$.

Simplification Rule: *(Works only if we have constant many terms.)*

1. Replace constant factors with 1,
2. transform complicated terms to simpler forms, and
3. remove any term that grows slower than another.

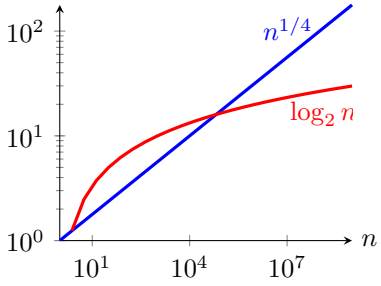
Use the rules below to simplify terms and compare their growth rate.

Example: $\Theta(9999 + 0.1 \cdot 2^n + 100 \cdot n^2 \cdot n) = \Theta(\mathbf{1} + \mathbf{1} \cdot 2^n + \mathbf{1} \cdot n^2 \cdot n) = \Theta(1 + 2^n + \mathbf{n^3}) = \Theta(\cancel{1} + 2^n + \cancel{n^3}) = \Theta(2^n)$

"LOG < POL" Rule:
Any logarithmic function grows slower than any polynomial function.
Formally: If a, b and c are positive constants, then there is a constant n_0, such that for all $n \geq n_0$, we have

$$a \log_b n < n^c \qquad \text{and} \qquad (\log_b n)^a < n^c \ .$$

For example, $\log_2 n < n^{1/4}$ *for sufficiently large* n*. Without the rule above, this fact is not obvious! If you try out different values for* n*, even as large as* $n = 10000$*, you might get the impression that* $\log_2 n$ *is growing faster. However, for all* $n \geq 65537$*, the function* $\log_2 n$ *is smaller than* $n^{1/4}$*.*



"POL < EXP" Rule:
Any polynomial function grows slower than any exponential function.
Formally: If a and $b > 1$ are constants, then there is a constant n_0, such that for all $n \geq n_0$, we have

$$n^a < b^n \ .$$

For example, $n^{100} < 1.01^n$ *for sufficiently large* n*.*

Transformation Rules:
For any a, b, c, we have

- $a^b \cdot a^c = a^{b+c}$ and $(a^b)^c = a^{b \cdot c}$,
- $\log_a(b \cdot c) = \log_a b + \log_a c$ and $\log_a(b^c) = c \cdot \log_a b$ (for positive variables)
- $a = b^{\log_b a}$ *definition of the logarithm* (for positive variables)
- $\Theta(\log_a c) = \Theta(\log_b c) = \Theta(\log c)$ (if a and b are constants).

Algorithms for Data Science
Exercise 2
(with Solutions)

See the cheatsheet for valuable remarks!

In our course: log *always means* $\log_2$.

Task 2.1: *Oh-Notation*

(a) Someone tells you that an algorithm A has running time $O(n \log n)$ and an algorithm B has running time $O(n^2)$. Do you know which algorithm is faster?

> **Solution:** The answer is no. You don't know it. You only know that the running times of the two algorithms are asymptotically not higher than what is given in the O-notations. It is exactly the same situation if someone told you that one building a is not higher than 10 meters, whereas another building b is not higher than 15 meters. You cannot tell which building is higher or if they are of equal height.
>
> If the running times of A and B had been given in Θ notation, then we could compare them and conclude, that A with $\Theta(n \log n)$ is asymptotically faster than B with $\Theta(n^2)$. (Why? By the "LOG $<$ POL" Rule from the Cheatsheet, $\log n = O(n)$ and, thus, $n \cdot \log n = O(n \cdot n) = O(n^2)$.) Note that we could also come to the same conclusion if the running time of A had been given as $\Omega(n \log n)$ with the running time of B given as $O(n^2)$.

Simplify the following running times. Explain the simplifications.

Difficult sample tasks explained by the tutor.

(b) $\Theta(10 \cdot n^2 + 978 \cdot n + n \cdot n \cdot 2 \cdot n/1000)$

Solution: $\Theta(n^3)$.
Formal proof: First observe that $n \cdot n \cdot 2 \cdot n/1000 = n^3/500$. Next, separately show that $10n^2 + 978 \cdot n + n^3/500$ is $O(n^3)$ and $\Omega(n^3)$.

- $\Omega(n^3)$ as $10n^2 + 978 \cdot n + n^3/500 \geq n^3/500$, and, therefore, by picking the constants $C = 1/500$ (or any smaller value) and $n_0 = 0$, we obtain, for all $n > n_0$ that $10n^2 + 978 \cdot n + n^3/500 \geq C \cdot n^3$.
- $O(n^3)$ as

$$\begin{aligned} & 10n^2 + 978 \cdot n + n^3/500 \\ & \leq 10n^3 + 978 \cdot n^3 + n^3/500 \\ & = (10 + 978 + 1/500)n^3 \\ & \leq 979.002 \cdot n^3 \ , \end{aligned}$$

  and, therefore, by picking the constants $C = 979.002$ (or any larger value, e.g., 1000) and $n_0 = 0$, we obtain, for all $n > n_0$ that $10n^2 + 978 \cdot n + n^3/500 \leq C \cdot n^3$.

Informal proof (using the simplification rule from the cheatsheet):

1. Remove constant factors: $\Theta(n^2 + n + n \cdot n \cdot n)$.
2. Transform complicated terms: $\Theta(n^2 + n + n^3)$.
3. Choose the fastest growing term: n^3.

(c) $\Theta(1000(\log_5 n)^{1000} + \sqrt{n} \cdot 2000 + (11/10)^n)$

Solution: $\Theta(1.1^n)$
Informal proof:

1. Remove constant factors: $\Theta((\log_5 n)^{1000} + \sqrt{n} + (11/10)^n)$.
2. Transform complicated terms: $\Theta((\log_5 n)^{1000} + \sqrt{n} + 1.1^n)$.
3. Choose the term that grows the fastest: 1.1^n.
   Explanation: By the "LOG < POL" rule (see the cheatsheet), $(\log_5 n)^{1000}$ grows slower than the polynomial $\sqrt{n} = n^{1/2}$, so we can remove the first term. By the "POL < EXP" rule, $\sqrt{n}$ grows slower than the exponential 1.1^n, so we remove also $\sqrt{n}$, which leaves us with only one term: 1.1^n.

(d) $\Theta(4 \cdot n^{1/4} \cdot n^{0.25} + 2(2^n)^{2/n} + 50 \log_2(100n) + \log_3(9^n))$

Solution: $\Theta(n)$
Informal proof:

1. Remove constant factors: $\Theta(n^{1/4} \cdot n^{0.25} + (2^n)^{2/n} + \log_2(100n) + \log_3(9^n))$.

2. Transform complicated terms: $\Theta(n^{1/2} + 1 + \log_2 n + n)$
   Explanation: Apply the transformation rules of the cheatsheat.
   - $n^{1/4} \cdot n^{0.25} = n^{1/4} \cdot n^{1/4} = n^{1/4+1/4} = n^{2/4} = n^{1/2}$.
   - $(2^n)^{2/n} = 2^{n \cdot 2/n} = 2^2 = 4$.
   - $\log_2(100n) = \log_2 100 + \log_2 n$.
   - $\log_3(9^n) = n \log_3 9 = n \cdot 2$.

   Thus, we obtain $\Theta(n^{1/2} + (4 + \log_2 100) + \log_2 n + 2n)$. By further simplifying constant factors, we get $\Theta(n^{1/2} + 1 + \log_2 n + n)$.
3. Choose the term that grows the fastest: n
   Explanation:
   By the "LOG < POL" and "POL < EXP" rule, the fastest growing term is n.

Regular tasks for the students.

(e) $\Theta(91n + n^2 \cdot 4n^2 + 3 \log n + 42n^3)$

Solution: $\Theta(n^4)$.
Informal proof:

1. Remove constant factors: $\Theta(n + n^2 \cdot n^2 + \log n + n^3)$
2. Transform complicated terms: $\Theta(n + n^4 + \log n + n^3)$
3. Choose the fastest growing term: n^4.

Formal proof: We have to separately show that $91n + 4n^4 + 3 \log n + 42n^3$ is $O(n^4)$ and $\Omega(n^4)$.

- $\Omega(n^4)$ as $91n + 4n^4 + 3 \log n + 42n^3 \geq n^4 = C \cdot n^4$ for $C = 1$ and for all $n \geq n_0 = 1$.
- $O(n^4)$ as

$$\begin{aligned} & 91n + 4n^4 + 3 \log n + 42n^3 \\ & \leq 91n^4 + 4n^4 + 3n^4 + 42n^4 \\ & = (91 + 4 + 3 + 42)n^4 \\ & = C \cdot n^4 \end{aligned}$$

  for $C = 91 + 4 + 3 + 42 = 139$ and all $n \geq n_0 = 1$.

(f) $\Theta(1000^{1000^{1000}} + 1/2 \cdot 1.0001^n + 40 \cdot n^{100})$

Solution: $\Theta(1.001^n)$
Informal proof:

1. Remove constant factors (or replace with 1): $\Theta(1 + 1.0001^n + n^{100})$.
2. Transform complicated terms: *No need, the terms are already simple.*
3. Choose the fastest growing term: 1.0001^n.
   Explanation: Obviously, it is not 1. By the "POL < EXP" rule, 1.0001^n is growing faster than n^{100}.

(g) $\Theta(5\log(5n))$.

> **Solution:** $\Theta(\log n)$ using $\log(5n) \geq \log n$ and $\log(5n) \leq \log(n^2) = 2\log n$ (for $n \geq 5$). (We applied a transformation rule from the cheatsheet.)

(h) $\Theta(5 \cdot 2^{5n+5})$.

> **Solution:** $\Theta(2^{5n})$ (or $\Theta(32^n)$) using that $2^{5n+5} = 2^5 \cdot 2^{5n} = \Theta(2^{5n})$. We cannot simplify $\Theta(2^{5n})$ further to $\Theta(2^n)$ as $2^{5n} = O(2^n)$ would imply that there are constants $c > 0$ and $n_0 \geq 0$ such that, for all $n \geq n_0$, $2^{5n} \leq c \cdot 2^n \Leftrightarrow 2^{4n} \leq c$; a contradiction.
> Another way to see it: $2^{5n} = (2^5)^n = 32^n$, which grows much faster than 2^n.

(i) $\Theta(42m + k/100 + m \cdot 21 \cdot k + 2^{2\log_2 n})$. *Sometimes there is more than one parameter.*

> **Solution:** $\Theta(mk + n)$
> Informal proof:
>
> 1. Remove constant factors: $\Theta(m + k + mk + 2^{2\log_2 n})$.
> 2. Transform complicated terms: $\Theta(m + k + mk + n^2)$.
>   Explanation: by the transformation rules from the cheatsheet, we have $2^{2\log_2 n} = (2^{\log_2 n})^2$, which, by the definition of the logarithm, is equal to $(n)^2 = n^2$.
> 3. Choose the fastest growing terms (not growing slower than another term): mk and n^2.
>   Explanation: m and k grow slower than mk. Both mk and n^2 do not grow slower than any other term, especially, they are incomparable to each other, so we take both. (We would need additional knowledge on the relationship of n, k and m in order to be able to simplify the running time further.)

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 2.2: *Oh-Notation++*

(a) You are given the following running time functions (in Θ-notation). Order them by order of growth!

$$n! \quad (\log n)^2 \quad n^2 \quad \sqrt{n} \quad 2^{\sqrt{2}\log n} \quad 2^{2n} \quad n \cdot 2^n \quad \log(n^2) \quad 4^{\log n}$$

> **Solution:**
>
> 1. $\log(n^2) = 2\log n$
> 2. $(\log n)^2$
> 3. $\sqrt{n}$
> 4. $2^{\sqrt{2}\log n} = n^{1.5\ldots}$
> 5. n^2 and $4^{\log n} = 2^{2\log n} = n^2$

6. $n \cdot 2^n$

7. $2^{2n} = 2^n \cdot 2^n$ (and therefore larger than the function $n \cdot 2^n$ above)

8. $n! \approx n^n$
   For our course, it suffices to remember that $n!$ grows almost as fast as n^n which is much faster than any exponential function. The following discussion offers optional enrichment for the curious student and is ***not mandatory*** *for our course:*
   To be precise, $n!$ grows slower than n^n ($n! \neq \Omega(n^n)$), however, people have shown that $n! \geq n^{0.9999n}$ for sufficiently large n.
   For our task, we just have to show that $n!$ grows faster than the exponential function 2^{2n}.
   We can do it as follows:
   By definition, $n! = 1 \cdot 2 \cdot \ldots \cdot (n-1) \cdot n$. Thus,

$$\begin{aligned} n! &> n/2 \cdot (n/2+1) \cdot \ldots \cdot (n-1) \cdot n \\ &> (n/2) \cdot \ldots \cdot (n/2) \\ &= (n/2)^{n/2} \\ &= (n/2)^{2n/4} \ . \end{aligned}$$

   If we assume that $n \geq 32$, we can replace the n in the base by 32, that is, we can write

$$(n/2)^{2n/4} \geq (32/2)^{2n/4} = 16^{2n/4} = (16^{1/4})^{2n} = 2^{2n} \ .$$

   (We used the transformation rule and the fact that $2^4 = 16$, that is, $16^{1/4} = 2$.) Hence, $n! > 2^{2n}$ for all $n \geq 32$.

Prove your answers for the following questions.

(b) Is $f(n) + g(n) = \Theta(\max(f(n), g(n))$? (Assume that f and g are nonnegative functions.)

Solution: Yes!
First observe that $f(n) + g(n) \geq \max(f(n), g(n)) = \Omega(\max(f(n), g(n))$.
Next, observe that, $f(n) + g(n) \leq 2\max(f(n), g(n)) = O(\max(f(n), g(n))$.
In total, Θ holds as O and Ω hold.

(c) Is $(n+100)^5 = \Theta(n^5)$?

Solution: Yes!
First observe that $(n+100)^5 \geq n^5 = \Omega(n^5)$.
Next, observe that, $(n+100)^5 \leq (n \cdot 100)^5 = n^5 \cdot 100^5 = O(n^5)$ as 100^5 is a constant.
In total, Θ holds as O and Ω hold.

(d) Is $\log_3 n + \log_5 n = \Theta(\log_7 n)$?

Solution: Yes!
By the transformation rules from the cheatsheet, $\log_3 n = \Theta(\log_7 n)$ and $\log_5 n = \Theta(\log_7 n)$.
Thus, $\log_3 n + \log_5 n = \Theta(\log_7 n + \log_7 n) = \Theta(\log_7 n)$,
If you are a curious student, try to prove $\log_3 n = \Theta(\log_7 n)$ yourself using the other transformation rules! (The proof is beyond the scope of the course.)

Continue to read only, if you have tried to prove it yourself!

$$\begin{aligned}\log_3 n &= \log_3(7^{\log_7 n}) \quad \text{(By the definition of the logarithm, } n = 7^{\log_7 n}\text{)}\\ &= \log_7 n \cdot \log_3 7 \quad \text{(Transformation rule)}\\ &= \Theta(\log_7 n) \quad \text{(Removing the constant factor } \log_3 7.\text{)}\end{aligned}$$

(e) Is $300\log(n^n k) + k^k + 2^{\sqrt{k}\log k} = O(n\log n + 2^{k\log k})$?

Solution: Yes!
We use the following rules:

- $\log(ab) = \log a + \log b$
- $\log(a^b) = b\log a$
- $a = 2^{\log a}$ and in particular $a^b = 2^{b\log a}$

Before continuing with reading, try to solve it yourself using the rules above!
Using the rules above and the fact that k^k grows faster than $k^{\sqrt{k}}$ and than $\log k$, we obtain

$$\begin{aligned}300\log(n^n k) + k^k + 2^{\sqrt{k}\log k} &= 300(\log(n^n) + \log k) + k^k + k^{\sqrt{k}}\\ &= 300n\log n + 300\log k + k^k + k^{\sqrt{k}}\\ &\leq 300n\log n + 300k^k + k^k + k^k\\ &\leq 303(n\log n + k^k)\\ &= 303(n\log n + 2^{k\log k})\\ &= O(n\log n + 2^{k\log k})\end{aligned}$$

Task 2.3: *Oh-Notation Definitions*

(a) In the lecture, we have seen running times depending on two parameters. For instance $O(kn)$. Propose a formal definition for $f(n_1,\ldots,n_i) = O(g(n_1,\ldots,n_i))$.

Solution: $f(n_1,\ldots,n_i) = O(g(n_1,\ldots,n_i))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that

$$f(n_1,\ldots,n_i) \leq c \cdot g(n_1,\ldots,n_i)$$

for all $n_1,\ldots,n_i$ that satisfy $n_j \geq n_0$ for each $j \in \{1,\ldots,i\}$. (Note that n_0 is our fixed constant, whereas $n_1,\ldots,n_i$ are the parameters of the functions.)

(b) Prove that $f(n) = \Omega(g(n))$ holds if $g(n) = O(f(n))$ using the definition from the cheatsheet.

Solution: Just look at the definition of O and divide by C. Define $1/C$ as a new constant C' and now observe that the definition of Ω is fulfilled.

Task 2.4: *Designing Algorithms: Airfield*

Motivation: You want to build an airfield in a mountainous area. Therefore, you look for a longest possible plateau, that is, for a longest possible flat surface. An additional constraint is that the airfield has to be build along a fixed line of latitude since this is the direction in which the airplanes land or take off. To solve this problem, you first measure the height of the surface along the line of latitude at n equidistant sample points.

Now, given an integer array $h[1..n]$ with the measured heights (rounded to full meters), propose an efficient algorithm to find a longest plateau, that is, a longest consecutive subsequence $h[i..j]$ of the array where all the integers have the same value. The output should consist of a position i of the array where a longest plateau starts.

What is the running time of your algorithm?

Solution: $\Theta(n)$ time: Go from left to right and compare the current plateau with the longest found so far.

Algorithm 1: LongestPlateau

```
Input: array h[1..n] of integers
Output: start position of a longest plateau
pos = 1 // The starting position of the longest plateau found so far.
len = 1 // The length of the longest plateau found so far.
cPos = 1 // The starting position of the current plateau.
cLen = 1 // The length of the discovered part of the current plateau.
for i = 2 to n do
    if h[i − 1] == h[i] then
        cLen++
        if cLen > len then
            pos = cPos
            len = cLen
    else
        cPos = i
        cLen = 1
return pos
```

Task 2.5: *Designing Algorithms: Forest Age / Salary Database*

(a) *Motivation 1 (Forest): Suppose that we have a database d with the age of all n trees of a forest (possibly obtained by some novel age-estimation technique using drones), where d[i] is the age of the i-th tree. We also received m e-mails from different biologists, each one asking how many trees are there of a specific age. We store all the queries in a table q, where, for the j-th e-mail that we received, we store in q[j] the age for which we have to compute the number of trees. Since the biologists are only interested in young trees, we know also that all values in q are smaller than some given small number k. (We assume that all the ages in q and d are rounded to full days.) Your task is to answer all the e-mails!*

Motivation 2 (Salary): You maintain a database storing the yearly salary of all n persons in Poland. The database is implemented as an integer array d[1..n] where d[i] is the salary of the i-th person in Polish Złoty (PLN). Now, your boss gives you a small database containing the salary (in PLN) of some other m persons whose salary is not larger than k PLN, also implemented as an integer array q[1..m]. Your task is to compute, for each person of the databse q, how many persons in the database d have exactly the same salary.

Input: arrays $d[1..n]$ and $q[1..m]$ of nonnegative integer values, and a parameter k with the guarantee that no value in q is larger than k.
Output: an array $res[1..m]$ where $res[i]$ is the number of indices $j \in \{1, \ldots, n\}$ for which $q[i] = d[j]$.

Propose an algorithm solving this problem using only $O(1)$ additional memory space, that is, besides d, q, k and res, you are allowed to use only a constant number of variables.

What is the running time of your algorithm?

Solution: Time: $\Theta(n \cdot m)$: For each entry in q, check how many same entries are there in d.

Input: arrays $d[1..n]$ and $q[1..m]$
Output: array $res[1..m]$ where $res[i]$ is the number of times $q[i]$ appears in d.
$res[1..m]$ = new array filled with zeros
for $i = 1$ **to** m **do**
  for $j = 1$ **to** n **do**
    if $q[i] == d[j]$ **then**
      $res[i]++$
return res

(b) We consider the same problem as in (a). This time, propose an algorithm with running time $O(n + m + k)$ without any constraints regarding the memory space.

What is the extra memory used by your algorithm in Θ notation?

Compare your running time to (a) when assuming $m = \Theta(\sqrt{n})$ and $k = \Theta(m)$.

Hint: Use the counting technique.

Solution: Create an auxiliary array $counts[1..k]$. Go through d: for each i, if $d[i] \leq k$, then increase $counts[d[i]]$ by one. Then, go through q and for each i, look the number up in $count[q[i]]$.

Input: arrays $d[1..n]$ and $q[1..m]$ where all values in q are bounded by k
Output: array $res[1..m]$ where $res[i]$ is the number of times $q[i]$ appears in d.

$counts[1..k]$ = new array filled with zeros `// It takes` $\Theta(k)$ `time to create and fill this array.`
for $j = 1$ **to** n **do**
  if $d[j] \leq k$ **then**
    `// Otherwise the value` $d[j]$ `is not interesting for us.`
    $counts[d[j]]++$
$res[1..m]$ = new array filled with zeros
for $i = 1$ **to** m **do**
  $res[i] = counts[q[i]]$
return res

Extra space usage: $\Theta(k)$ for creating the auxiliary array.

Comparison with the first algorithm: If $m = \Theta(\sqrt{n})$ and $k = \Theta(m)$, then the running time of the first algorithm is $\Theta(n\sqrt{n})$ whereas the running time of the second algorithm is $\Theta(n)$.

Task 2.6: *Analyzing Algorithms: SomeSortingAlgo*

Algorithm 2: SomeSortingAlgo

Input: array $a[1..n]$ of integers
Output: array a is sorted non-decreasingly

```
1  i = 1
2  while i ≤ n − 1 do
3      minPos = i
4      j = i + 1
5      while j < n do
6          if a[j + 1] < a[minPos] then
7              minPos = j + 1
8          j++
9      if minPos ≠ i then
           // exchanges the values at position i and minPos in the array a
10         Swap(a,i,minPos)
11     i++
```

(a) Write an algorithm for the function `Swap(`a`,`i`,`ℓ`)` that exchanges the values at position i and ℓ in the array a, that is, after calling `Swap(`a`,`i`,`ℓ`)`, $a[i]$ contains the former value at $a[\ell]$, and $a[\ell]$ contains the former value at $a[i]$.

Solution:

Algorithm 3: Swap(a, i, ℓ)

Input: An array $a[1 \dots n]$ and two indices i and ℓ with $1 \leq i, \ell \leq n$.
Output: The array a with the elements at positions i and ℓ swapped.

```
1  Swap(a, i, ℓ)
2      temp = a[i]
3      a[i] = a[ℓ]
4      a[ℓ] = temp
```

(b) Give the best case and the worst case running times in Θ-notation of `SomeSortingAlgo`.

Solution: Best=Worst: $\Theta(\sum_i (n-i)) = \Theta(n^2)$
Explanation:
The first line runs in constant time, but its contribution is negligible due to subsequent code.
In each iteration of the outer loop ($i = 1$ to $n-1$), the inner loop runs from $j = i+1$ to $j = n$, resulting in exactly $n - i$ iterations. As each iteration takes constant time, the total time for the inner loop in the i-th outer loop iteration is $\Theta(n - i)$.
The remaining operations in the outer loop run in constant time and are dominated by the inner loop. Summing $\Theta(n-i)$ over all i from 1 to $n-1$ gives the final running time, as detailed in the formula analysis that follows.

$$
\begin{aligned}
&= \Theta(n-1) \;+\; \Theta(n-2) \;+\; \ldots \;+\; \Theta(n-(n-1)) && = \sum_{i=1}^{n-1} \Theta(n-i) \\
&= \Theta\left((n-1) \;+\; (n-2) \;+\; \ldots \;+\; (n-(n-1))\right) && = \Theta\left(\sum_{i=1}^{n-1} (n-i)\right) \\
&= \Theta\left((n-1) \;+\; (n-2) \;+\; \ldots \;+\; 1\right) \\
&= \Theta\left(1 \;+\; 2 \;+\; \ldots \;+\; (n-1)\right) && = \Theta\left(\sum_{i=1}^{n-1} i\right) \\
&= \Theta\left(\frac{n(n-1)}{2}\right) && \text{(See cheatsheet)} \\
&= \Theta\left(\frac{n^2}{2} \;-\; \frac{n}{2}\right) \\
&= \Theta(n^2)
\end{aligned}
$$

(c) The algorithm has a mistake. Give a shortest possible input array for which the algorithm computes a wrong output. Describe the mistake and fix it by changing the code as little as possible.

Solution: One of many possible answers: If $a = \langle 2, 1\rangle$, then the output of the algorithm is also $\langle 2, 1\rangle$ (which is wrong); the correct output would be $\langle 1, 2\rangle$.

The problem is that the comparison "$a[j+1] < a[minPos]$?" within the inner while loop never compares $a[i]$ with $a[i+1]$. Though $minPos = i$ in the beginning, we always have $j+1 \geq i+2$ due to the mistake that we start with $j = i+1$ in line 4. Consequently, if $a[i+1]$ is the smallest element in $a[i..n]$, we will not notice this fact and we will not move $a[i+1]$ to its correct position i.

To fix the error, it suffices to drop the "+1" in line 4, so that initially we know have $j = i$. The full modified code (changes in green):

Algorithm 4: SomeSortingAlgo

```
   Input: array a[1..n] of integers
   Output: array a is sorted non-decreasingly
1  i = 1
2  while i ≤ n − 1 do
3  |  minPos = i
4  |  j = i  // Repaired line
5  |  while j < n do
6  |  |  if a[j + 1] < a[minPos] then
7  |  |  |  minPos = j + 1
8  |  |  j++
9  |  if minPos ≠ i then
   |  |  // exchanges the values at position i and minPos in the array a
10 |  |  Swap(a,i,minPos)
11 |  i++
```

(d) Prove that in lines 3 to 8, the algorithm that you repaired in task (c) finds a position $minPos$ of a smallest element in $a[i..n]$ (that is, $a[minPos]$ is this smallest element). For your proof, come up with a helpful loop invariant.

Solution:

Invariant (with respect to line 5): "$a[minPos]$ *is a smallest element in* $a[i..j]$."

In the beginning, $minPos = j = i$, thus the subarray $a[i..j] = a[i..i]$ contains only the element $a[i] = a[minPos]$ and the claim of the invariant is trivially true.

Now assume that the claim is true for some j at line 5. If $a[j+1] \not< a[minPos]$, then $a[j+1] \geq a[minPos]$ and, obviously, $a[minPos]$ is also a smallest element in $a[i..j+1]$. Since we do not change $minPos$ in this case, the claim remains true for $j+1$. If $a[j+1] < a[minPos]$, then, by our assumption, $a[j+1]$ is smaller than the smallest value in $a[i..j]$ and, hence, $a[j+1]$ is the smallest element in $a[i..j+1]$. Thus, after setting $minPos = j+1$ in line 7, the claim is true for $j+1$.

The above arguments prove that our invariant is correct.

At the end, when we exit the while loop, we have $j = n$. Since our invariant is true, we conclude that $minPos$ is a position of a smallest element in $a[i..j] = a[i..n]$.

(e) Prove the correctness of the whole algorithm `SomeSortingAlgo` via the following invariant (defined with respect to line 2):

The subarray $a[1..i]$ *is sorted non-decreasingly, and there is no element in the subarray* $a[1..i-1]$ *that is larger than some element in* $a[i..n]$. *Furthermore* $a[1..n]$ *contains all the elements that it contained in the beginning.* (For $i = 1$, we define $a[1..i-1] = a[1..0]$ as an empty array.)

Recall that you have to prove that the invariant holds (i) in the beginning (when $i = 1$), and (ii) that it holds for each $i > 1$ using the assumption that it holds for $i - 1$ (equivalently: that it holds for $i + 1$ assuming that it holds for $i \geq 1$). Finally, use the invariant to conclude that at the end the whole input array is sorted.

Solution:

In the beginning (when we reach line 2 for the first time), $i = 1$ and therefore $a[1..i] = a[1]$ which is obviously a sorted array. Since $a[1..i-1] = a[1..0]$ is an empty array, there is indeed no element in $a[1..i-1]$ that is larger than some element in $a[i..n]$. Thus, the invariant holds for $i = 1$.

Now, assume that the invariant holds for some i with $1 \leq i \leq n-1$ at line 2. In lines 3 to 8, the algorithm finds a position $minPos$ of a smallest element in $a[i..n]$ (that is, $a[minPos]$ is this smallest element). We proved this fact in the subtask (d) above. In the next two lines, the algorithm exchanges this smallest element with the element that is on position i (provided that not already $i = minPos$). Thus, in the beginning of line 11, $a[i]$ contains a smallest element from $a[i..n]$, and $a[i..n]$ contains all the elements that it contained before. Since the invariant holds for i, we know that $a[1..i-1]$ is sorted non-decreasingly and that there is no element in $a[1..i-1]$ that is larger than some element in $a[i..n]$. Consequently, since $a[i]$ is a largest element from $a[1..i]$ and a smallest one from $a[i..n]$, we know that $a[1..i+1]$ also sorted non-decreasingly and $a[1..i]$ contains no element larger than some element in $a[i+1..n]$. Thus the invariant holds also for $i+1$ in the beginning of line 11, and, after increasing i by one, the invariant also holds for the next i at line 2.

Now let us use the invariant to show that the algorithm is correct. At the end, when the program leaves the while loop at line 2, the while loop condition has to be violated. Thus, at the end, we have $i > n-1$, which in our algorithm can be only if $i = n$. By putting $i = n$ into the definition of our invariant, we obtain that a contains all the elements as in the beginning and that $a[1..i] = a[1..n] = a$ is sorted non-decreasingly. Hence, our algorithm is correct!

Task 2.7: *Analyzing Algorithms: StrangeAlgo*

Algorithm 5: StrangeAlgo

Input: nonnegative integers n and k
Output: ?

```
1  StrangeAlgo(n,k)
2      if k == 0 then
3          return 0
4      if k == 1 then
5          return 1
6      a[1..1000] = new array
7      i = 1
8      while i ≤ 1000 do
9          a[i] = i
10         i++
11     while i ≤ n do
12         i++
13     ret = StrangeAlgo(n,k − 2)
14     return ret
```

(a) What is the output of `StrangeAlgo(`1000000000000000*,* 8941307`)`?

Solution: The output is 1 as the algorithm computes $k \mod 2$, that is, it returns 0 if k is even, and 1 if k is odd. To see this, observe that at each call of `StrangeAlgo`, k is decreased by exactly 2. Thus, at the last call, we have $k = 1$ or $k = 0$, depending whether originally k was odd or even.

(b) What is the running time of `StrangeAlgo(`n*,* k`)` in Θ-notation in dependence of n and k?

Solution: $\Theta(n \cdot k)$—`StrangeAlgo` is called $\lceil k/2 \rceil$ times, each time performing $\Theta(n)$ operations. A more detailed proof:
`StrangeAlgo`(n, k) calls `StrangeAlgo`$(n, k-2)$, which calls `StrangeAlgo`$(n, k-4)$, and so on, until reaching `StrangeAlgo`$(n, 1)$ or `StrangeAlgo`$(n, 0)$. Thus, `StrangeAlgo` is called $\lceil k/2 \rceil$ times. The final call takes constant time, while each previous call takes $\Theta(n)$ time: the second while loop ($i \leq n$) runs in $\Theta(n)$, and all other lines run in constant time. In particular, the first while loop ($i \leq 1000$) executes at most 1000 times, which is constant. Hence, the total time is $\Theta\left((\lceil k/2 \rceil - 1)\, n + 1\right) = \Theta(n \cdot k)$.

(c) What is the memory complexity of `StrangeAlgo(`n*,* k`)` in Θ-notation in dependence of n and k?

Solution: $\Theta(k)$—At each call of `StrangeAlgo`, an array a of constant size 1000 is build. However, this array exists and is not discarded from memory when the next call occurs (it will be discarded only when the function call that has created this array makes its its return statement and ends). Thus, at the last call of `StrangeAlgo`—and there are $\lceil k/2 \rceil$ calls—the memory has to maintain $\lceil k/2 \rceil$ such constant arrays and some additional constant many variables (including some auxiliary invisible ones to keep track of the calls) for each call. Thus, the memory consumption in total is $\Theta(k)$.

(d) Modify `StrangeAlgo(`n`, `k`)` to run in time $\Omega(n^k)$.

Solution: A possible solution: We remove the lines

```
if k == 1 then
    return 1
```

and we replace the line $ret =$ `StrangeAlgo(`n`,`k`-2)` by the following code.

```
i = 1
while i ≤ n do
    StrangeAlgo(n,k − 1)
    i++
ret = k − (k/2) ∗ 2
```

The last line computes 1 if k is odd and 0 otherwise; recall that / is integer division. We need this last line if we want that the output of `StrangeAlgo` is the same as before—note that now we call `StrangeAlgo` with $k-1$ instead of $k-2$.

To see the running time, observe that there is one call with value k for k, and that there are n calls with value $k-1$, n^2 with value $k-2$, n^3 with value $k-3$, ..., and, finally, there are n^k calls with value 0. Since each of the last calls takes $\Omega(1)$ running time, we know that the total running time is at least $n^k \cdot \Omega(1) = \Omega(n^k)$ as claimed.

Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 2.8: *Cyclic Rotation*

Given an array $a[1..n]$ and an integer $k \in [1, n]$, rotate the elements in a by k elements to the left. In other words, at the end when your program stops, for every $1 \le i \le n$, $a[i]$ should contain the value that intially was at position $a[(i+k)\%n]$.

Find the following two solutions, both solutions with optimal asymptotic running time and $\Theta(1)$ extra space complexity!

(a) In the first solution, minimize the number of times you access the array; for this solution, you are allowed to call the function $\gcd(i, j)$ that returns in time $\Theta(\log^2 i + \log^2 j)$ the greatest common divisor of i and j.

Solution: Time: $\Theta(n)$.

For $i = 1$ up to $\gcd(n, kj)$ (compute the greatest common divisor only once at the beginning), save $a[i]$ in a *temp* variable and move $a[(i+k)\%n]$ to $a[i]$, then $a[(i+2k)\%n]$ to $a[(i+k)\%n]$, then $a[(i+3k)\%n]$ to $a[(i+2k)\%n]$ until it's time to move $a[i]$. Instead of $a[i]$ write the *temp* value on the corresponding position, and continue with the for loop.

The solution is not obvious and you have to use math to prove its correctness.

(b) For the second solution, the only thing you cannot access the array directly but you are allowed to call the function Reverse(a, i, j) that reverses the element order in the subarray $a[i..j]$ (in time $\Theta(j - i)$).

Solution: Time: $\Theta(n)$.

The solution is very elegant and simple but I don't want to spoil it to you. I will only tell you as much as this: it's a three-liner of the form:

```
Reverse(a, ?, ?)
Reverse(a, ?, ?)
Reverse(a, ?, ?)
```

Algorithms for Data Science
Cheatsheet for Exercises 3

Master Theorem. *Let* $\boldsymbol{T(n) = aT(n/b) + f(n)}$ *with constants* $a > 0$, $b > 1$, *and* $f(n)$ *being a nondecreasing function. Let* $c = \log_b a$.

- ***(Case A)*** *If* $f(n) = O(n^{c'})$ *for some* $c' < c$, *then* $\boxed{T(n) = \Theta(n^c)}$.
- ***(Case B)*** *If* $f(n) = \Theta(n^c)$, *then* $\boxed{T(n) = \Theta(n^c \log n)}$.
- ***(Case C)*** *If* $f(n) = \Omega(n^{c'})$ *for some* $c' > c$, *then*[(*)] $\boxed{T(n) = \Theta(f(n))}$.

[(*)]*Note: for Case C,* $f(n)$ *must also satisfy the* regularity condition*: there is a constant* $k < 1$ *such that for all sufficiently large* n, *we have* $af(n/b) \leq kf(n)$. *However, in our course, you don't have to learn it.*

It is very easy to decide which case holds if $f(n)$ *is a polynomial:*

Master Theorem "Lite"
Compute $c = \log_b a$ as above. If $f(n) = \Theta(n^d)$ for some constant number d such that

- $\ldots d < c$, then Case A holds.
- $\ldots d = c$, then Case B holds.
- $\ldots d > c$, then Case C holds.

Note: the regularity condition holds automatically for Case C if $f(n) = \Theta(n^d)$.

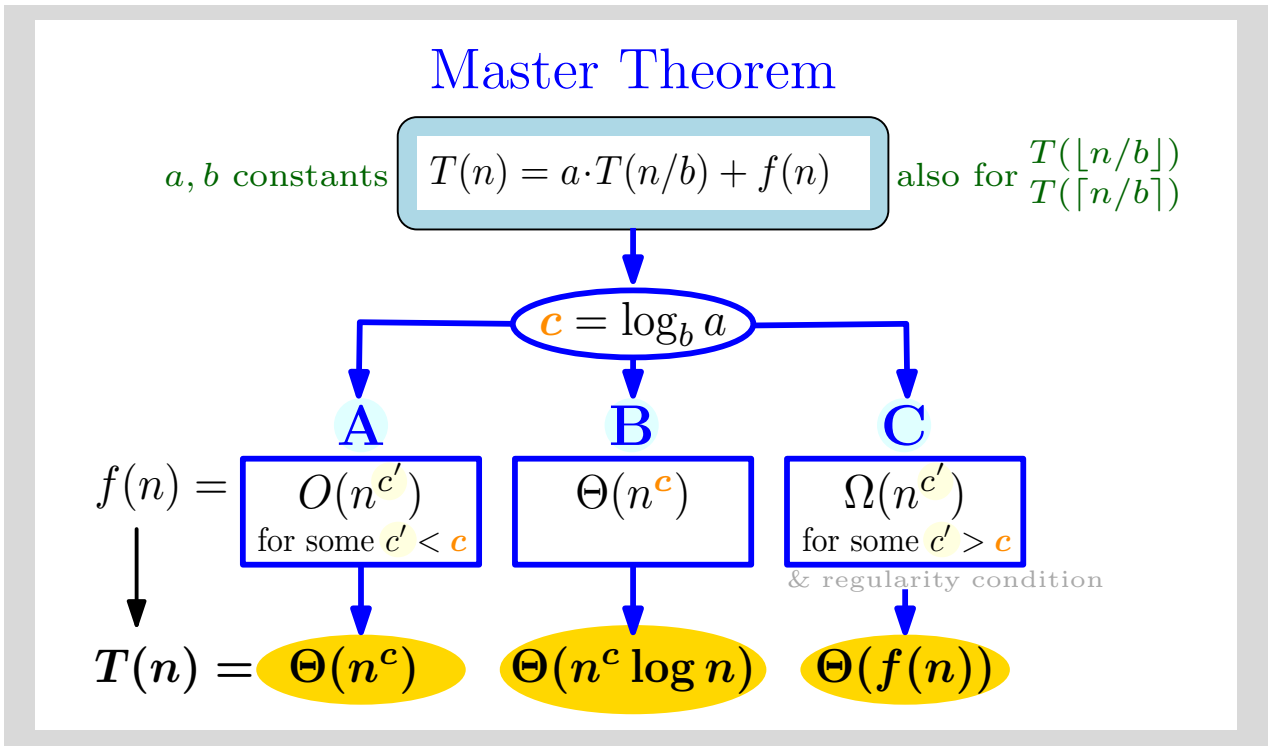
Nice to Know. *If* $T(n) = aT(\lfloor n/b \rfloor) + f(n)$ *or* $T(n) = aT(\lceil n/b \rceil) + f(n)$, *then we can simplify and write* $T(n) = aT(n/b) + f(n)$. *This simplification does not change the solution to* $T(n)$.

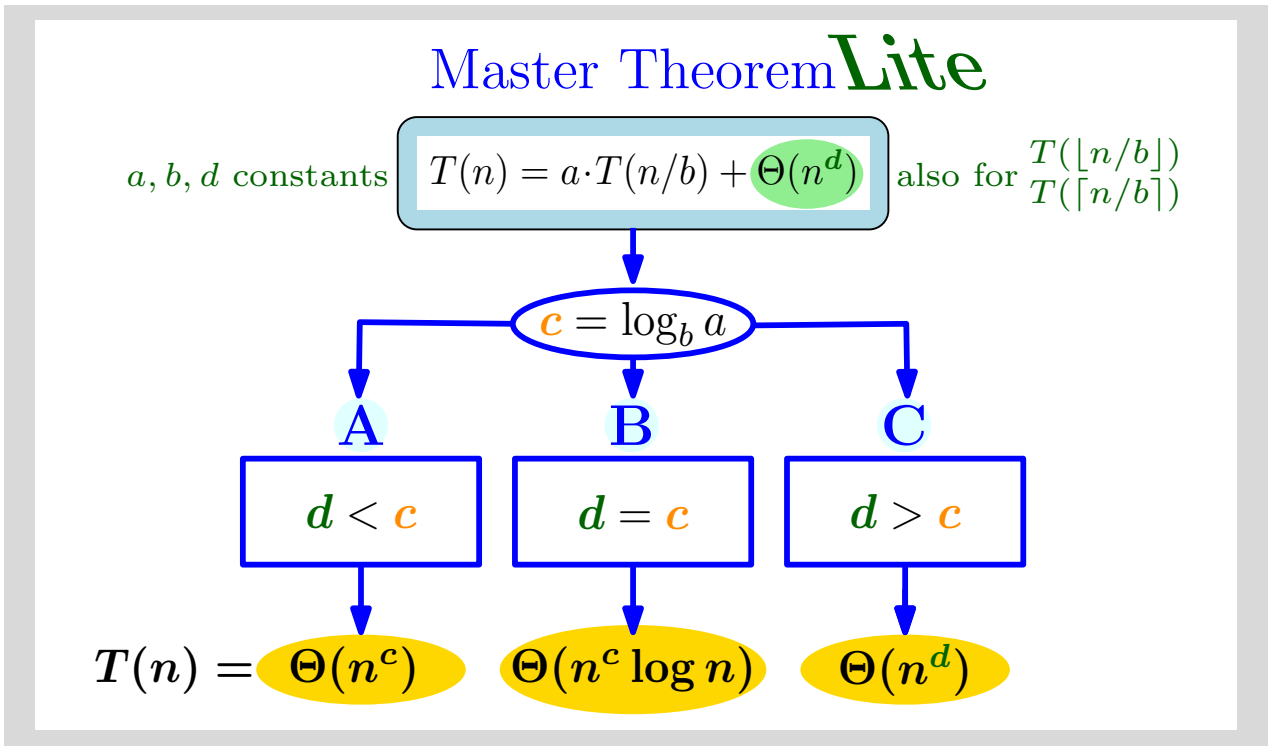
Notation. Recall that

- $\lfloor\ \rfloor$ is the *floor* function that rounds a given number *down* to the next integer.
  Examples: $\lfloor 1.5 \rfloor = 1$ *and* $\lfloor 4 \rfloor = 4$.
- $\lceil\ \rceil$ is the *ceiling* function that rounds a given number *up* to the next integer.
  Examples: $\lceil 1.5 \rceil = 2$ *and* $\lceil 4 \rceil = 4$.

Summation Rules.

1. (First n integers) $1 + \cdots + n = \sum_{i=1}^{n} i = n(n+1)/2 = \Theta(n^2)$
2. (First n power of twos) $1 + 2 + 2^2 + \cdots + 2^n = \sum_{i=0}^{n} 2^i = 2^{n+1} - 1 = \Theta(2^n)$


Master Theorem
a, b constants
$T(n) = a \cdot T(n/b) + f(n)$
also for $T(\lfloor n/b \rfloor)$ $T(\lceil n/b \rceil)$
$c = \log_b a$
A
B
C
$f(n) =$
$O(n^{c'})$ for some $c' < c$
$\Theta(n^c)$
$\Omega(n^{c'})$ for some $c' > c$
& regularity condition
$T(n) =$
$\Theta(n^c)$
$\Theta(n^c \log n)$
$\Theta(f(n))$


Master Theorem Lite
a, b, d constants
$T(n) = a \cdot T(n/b) + \Theta(n^d)$
also for $T(\lfloor n/b \rfloor)$ $T(\lceil n/b \rceil)$
$c = \log_b a$
A
B
C
$d < c$
$d = c$
$d > c$
$T(n) =$
$\Theta(n^c)$
$\Theta(n^c \log n)$
$\Theta(n^d)$

Algorithms for Data Science
Exercise 3
(with Solutions)

See the cheatsheet for valuable remarks!

Task 3.1: *Divide and Conquer*

Write a recurrence for the running time of the following algorithms in dependence of n. Write it in the form $T(n) = aT(n/b) + f(n)$.

Then solve the recurrences!

(a)

```
StrangeAlgo1(n)
    if n > 1 then
        i = 1
        while i ≤ n do
            i++
        StrangeAlgo1(n/2)
        StrangeAlgo1(n/2)
        StrangeAlgo1(n/2)
        StrangeAlgo1(n/2)
    return 1;
```

Solution: Recurrence: $T(n) = 4T(n/2) + \Theta(n)$

Solving the recurrence:
We have $a = 4$, $b = 2$, and $f(n) = \Theta(n)$. Thus, $c = \log_b a = \log_2 4 = 2$.

Determining the Case of the Master Theorem:

- Simpler approach with Master Theorem "Lite":
  We have $f(n) = \Theta(n) = \Theta(n^d)$ for $d = 1$. Since $d < c$, Case A holds.
- Alternative approach with normal Master Theorem:
  Let $c' = 1 < c$. Since $f(n) = O(n) = O(n^{c'})$, Case A holds.

Final solution: Case A $\longrightarrow T(n) = \Theta(n^c) = \boldsymbol{\Theta(n^2)}$.

(b) What is the running time of the algorithm above if we change the while loop condition from $i \leq n$ to $i \leq n \cdot \log_2 n$?

Solution: Now, the recurrence is $T(n) = 4T(n/2) + \Theta(n \log n)$. (Thus, we have $f(n) = \Theta(n \log n)$ instead of $f(n) = \Theta(n)$.)

Solving the recurrence:
We have the same values for a and b as before, thus, again, $c = 2$.

Case of the Master Theorem:
Since $f(n)$ is not a polynomial, we can't apply Master Theorem "Lite". We have to use the normal Master Theorem. We can't choose $c' = 1$ as before, since $n \log_2 n \neq O(n^1)$ (that is, $f(n)$ grows faster than n). Thus, we need to choose a c' within $1 < c' < 2$. To be on the safe side, let's take a c' very close to c, for instance, $c' = 1.999$. We have

$$n^{c'} = n^{1.999} = n^{1+0.999} = n \cdot n^{0.999} > n \log_2 n \ .$$

The last inequality holds as $n^{0.999} > \log_2 n$ for sufficiently large n (by the "LOG < POL" Rule from the Cheatsheet of Exercises 2). The inequality $n \log_2 n < n^{c'}$ immediately implies $f(n) = O(n^{c'})$. Hence, we are in case A (as before).

Final solution: Case A $\longrightarrow T(n) = \Theta(n^c) = \mathbf{\Theta(n^2)}$ (as before!).
Very interesting: The total running time did not change despite the fact that the while loop takes now significantly longer than before.

(c) And what if we change the while loop condition to $i \leq n^2/\log_2 n$ instead? Can we apply the Master Theorem?

Solution: The short answer: **no.**
The detailed answer: Now, $T(n) = 4T(n/2) + \Theta(n^2/\log_2 n)$. The constants a and b are the same before, thus $c = \log_b a$ is still 2. However, $f(n)$ changed to $\Theta(n^2/\log_2 n)$.
Is it Case B? That is, $f(n) = \Theta(n^c)$? Obviously, we have $f(n) = O(n^c)$ as $n^2/\log_2 n \leq n^2$ for all $n \geq 2$. Yet, $n^2 \neq O(n^2/\log_2 n)$ (and thus, $f(n) \neq \Omega(n^c)$) because if $n^2 = O(n^2/\log_2 n)$ were true, there would be a constant d such that, for all large enough n,

$$n^2 \leq d \cdot n^2/\log_2 n \rightarrow \log_2 n \leq d,$$

which, however, contradicts the assumption that d is a constant.
Therefore, only Case A might hold (given $f(n) = O(n^c)$). But does it? Suppose for a moment it does. Thus, there is a constant $c' < c = 2$ such that $f(n) = O(n^{c'})$, that is, there is another constant d such that $n^2/\log_2 n \leq d \cdot n^{c'}$ for all large n. Let $c'' = c - c'$ and observe that $n^{c'} = n^{c-c''} = n^2/n^{c''}$. Thus, we have:

$$n^2/\log_2 n \leq d \cdot n^2/n^{c''} \tag{1}$$

$$n^{c''} \leq d \cdot \log_2 n \ . \tag{2}$$

Since $n^{c''}$ is a polynomial (with positive constant degree c'' given $c' < c$), the inequality above contradicts the "LOG < POL" Rule from the Cheatsheet of Exercises 2. Consequently, Case A does not hold, which means, that **no case of the Master Theorem holds!** Thus, we cannot solve the recurrence using the Master Theorem.

So what can we do in such a situation? Luckily, there is an extended form of the Master Theorem (not discussed in our course) that yields the running time $T(n) = \Theta(n^2 \log\log n)$. To solve even more complicated running time recurrences, one can try the very general Akra-Bazzi Theorem.

(d)

```
StrangeAlgo2(n)
    if n > 1 then
        for i = 1 to n do
            for j = 1 to n do
                k = j
                while k ≤ n do
                    k = k + 1
        for i = 1 to 8 do
            StrangeAlgo2(n/2)
    return 1;
```

Solution: Recurrence: $T(n) = 8T(n/2) + \Theta(n^3)$
Why $+\Theta(n^3)$? Choose $k = k + 1$ as the dominating operation. Observe that it is executed

$$n \cdot \sum_{j=1}^{n} (n - j + 1) = n \cdot \sum_{j=1}^{n} j = n \cdot n \cdot (n+1)/2 = \Theta(n^3)$$

times in total.

Solving the recurrence:
We have $a = 8$, $b = 2$, and $f(n) = \Theta(n^3)$. Thus, $c = \log_b a = \log_2 8 = 3$.

Case of the Master Theorem:

- Simpler approach with Master Theorem "Lite":
  We have $f(n) = \Theta(n^3) = \Theta(n^d)$ for $d = 3$. Since $d = c$, we are in Case B.
- Alternative approach with the normal Master Theorem:
  Since $f(n) = \Theta(n^3) = \Theta(n^c)$, Case B holds.

Final solution: Case B $\longrightarrow T(n) = \Theta(n^c \log n) = \boldsymbol{\Theta(n^3 \log n)}$.

(e)

```
StrangeAlgo3(a[1..n])
    if n < 2 then
        return a[1]
    else
        ℓ = ⌊n/5⌋
        a' = a[1..ℓ]
        b' = a[2..ℓ + 1]
        p = StrangeAlgo3(a')
        a'[1] = p − 1
        return min(StrangeAlgo3(a'), StrangeAlgo3(b'))
```

Solution: Recurrence: $T(n) = 3T(\lfloor n/5 \rfloor) + \Theta(n)$ (simplified: $T(n) = 3T(n/5) + \Theta(n)$)

Solving the recurrence:
We have $a = 3$, $b = 5$, and $f(n) = \Theta(n) = \Theta(n^1)$. Thus, $c = \log_b a = \log_5 3 = 0.68\cdots < 1$. (You don't have to compute the logarithm, you can also use the obvious fact $\log_5 3 < \log_5 5 = 1$.)

Case of the Master Theorem:

- Simpler approach with Master Theorem "Lite":
  We have $f(n) = \Theta(n) = \Theta(n^d)$ for $d = 1$. Since $d > c$ and given that the regularity condition holds by the given assumption(*), we are in Case C.
- Alternative approach with the normal Master Theorem:
  Let $c' = 1 > c$. Since $f(n) = \Omega(n^{c'})$ and since the regularity condition holds by assumption(*), we conclude that Case C holds.

Final solution: Case C $\longrightarrow T(n) = \Theta(f(n)) = \boldsymbol{\Theta(n)}$.

(*): *A proof of the regularity condition is not required in this task. We are allowed to assume that the condition holds. For completeness, though, here a proof if you don't believe the assumption. The regularity condition holds if there is a constant c'' with $0 < c'' < 1$ such that $c'' \cdot f(n) \geq a \cdot f(n/b)$ (where $f(n)$ is the exact function $f(n) = n$ and not the whole "function class" $\Theta(f(n))$). To check if such a c'' exist, let us first compute $a \cdot f(n/b) = 3 \cdot f(n/5) = 3 \cdot n/5 = 3/5 \cdot f(n) \leq c'' \cdot f(n)$ for $c'' = 3/5 < 1$ as required. Hence, the regularity condition holds.*

In fact, the regularity condition automatically holds if $f(n)$ is a polynomial. This is the reason why Master Theorem "Lite" does not ask for the regularity condition in Case C.

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 3.2: *Master Theorem*

Solve the following recurrences using the master theorem. *First, determine $f(n)$ and the values of a and b. Secondly, compute c. Finally, conclude which case of the master theorem holds (by determining an appropriate c' if Case A holds) and write the solution.*

In the following, you can assume that $f(n)$ satisfies the regularity condition of Case C provided that the first condition of Case C holds.

(a)

$$T(n) = 5T(n/5) + \Theta(\log n)$$

Solution: We have $a = 5$, $b = 5$, and $f(n) = \Theta(\log n)$. Thus, $c = \log_b a = \log_5 5 = 1$. Let $c' = 1/2 < c$. Since $n^{1/2}$ grows faster than $\log n$, we have $f(n) = O(\log n) = O(n^{1/2}) = O(n^{c'})$. Hence, Case A holds. Thus, we have $T(n) = \Theta(n^c) = \mathbf{\Theta(n)}$.

(b)

$$T(n) = 3T(n/9) + \sqrt{n} + n^{1/4}$$

Solution: We have $a = 3$, $b = 9$, and $f(n) = \sqrt{n} + n^{1/4} = n^{1/2} + n^{1/4} = \Theta(n^{1/2})$. Thus, $c = \log_b a = \log_9 3 = 1/2$. Since $f(n) = \Theta(n^{1/2}) = \Theta(n^c)$, Case B holds. Thus, we have $T(n) = \Theta(n^c \log n) = \mathbf{\Theta(\sqrt{n} \log n)}$.

(c)

$$T(n) = 10^{20} T(\lceil n/100 \rceil) + 2^{\log(n^9)} \cdot \log n$$

Solution: We have $a = 10^{20}$, $b = 100$, and $f(n) = 2^{\log(n^9)} \cdot \log n = n^9 \log n$. Thus, $c = \log_b a = \log_{100} 10^{20} = \log_{100} 10^{2 \cdot 10} = \log_{100}(10^2)^{10} = \log_{100} 100^{10} = 10$. Let $c' = 9.5 < c$. Since $n^{9.5} = n^9 \cdot \sqrt{n}$, it grows faster than $n^9 \cdot \log n$. Thus, $f(n) = O(n^9 \log n) = O(n^{c'})$ and Case A holds. Therefore, we have $T(n) = \Theta(n^c) = \mathbf{\Theta(n^{10})}$.

(d)

$$T(n) = n + T(\lfloor n/9 \rfloor)$$

Solution: We have $a = 1$, $b = 9$ and $f(n) = n$. Thus, $c = \log_b a = \log_9 1 = 0$. Let $c' = 1 > c$. Since $f(n) = \Omega(n) = \Omega(n^{c'})$, Case C holds. Since by assumption also the regularity condition holds, we have $T(n) = \Theta(f(n)) = \mathbf{\Theta(n)}$.

(e)

$$T(n) = 14n + \sqrt{10000} - 4n + T(\lceil n/10 \rceil) - 10n$$

Solution: After simplifying, we have $T(n) = T(n/10) + 100$. Thus, we have $a = 1$, $b = 10$ and $f(n) = 100 = \Theta(1)$. Thus, $c = \log_b a = \log_{10} 1 = 0$. Since $n^c = n^0 = 1$, we have $f(n) = \Theta(1) = \Theta(n^c)$ and Case B holds. Thus, $T(n) = \Theta(n^c \log n) = \mathbf{\Theta(\log n)}$.

Task 3.3: *Simplifications of Recurrences*

Rewrite the following recurrences in the form $T(n) = aT(n/b) + \Theta(f(n))$ (where a and b are constants) simplifying $f(n)$ as far as possible. You don't have to solve the recurrences.

(a)

$$T(n) = 2T(n/3) + 42n^2 + 100n + T(n/3)$$

Solution: $T(n) = \boldsymbol{3T(n/3) + \Theta(n^2)}$
Proof: $f(n) = 42n^2 + 100n = \Theta(n^2)$.

(b)

$$T(n) = 13 + 20\log n + T(n/7) + 2(\sqrt{n} + T(n/7))$$

Solution: $T(n) = \boldsymbol{3T(n/7) + \Theta(\sqrt{n})}$
Proof: $f(n) = 2\sqrt{n} + 20\log n + 13 = \Theta(\sqrt{n})$. (Recall that $\sqrt{n}$ grows faster than $\log n$.)

(c)

$$T(n) = \log(n^3) + n^{\log_4 1} \cdot T(n/10) + (\log n)^2 + \log(1024) + \log(\log n)$$

Solution: $T(n) = \boldsymbol{T(n/10) + \Theta(\log^2 n)}$.

Note: $\log^2 n$ *is short for* $(\log n)^2$*, which in turn is short for* $(\log n) \cdot (\log n)$.

Proof: First, observe that $\log_4 1 = 0$. Thus, $n^{\log_4 1} = n^0 = 1$. Therefore, $n^{\log_4 1} \cdot T(n/10) = 1 \cdot T(n/10)$. Secondly, recall that $\log(n^3) = 3\log n$. Thus, $f(n) = \log^2 n + 3\log n + \log\log n + \log(1024)$. Note that $\log(1024)$ (which resolves to 10) is an uninteresting constant. The fastest growing function is $\log^2 n$. Thus, $f(n) = \Theta(\log^2 n)$ as claimed.

Task 3.4: *Divide and Conquer++*

For each of the following algorithms, write a recurrence for the running time and a recurrence for the *memory complexity*. Write the recurrences in the form $T(n) = aT(n/b) + \Theta(f(n))$ simplifying $f(n)$ as far as possible. You don't have to solve the recurrences. (For the memory complexity, use M instead of T.)

(a)

```
StrangeAlgo4(a[1..n])
    if n < 2 then
        return a[1]
    else
        ℓ = ⌊n/7⌋
        a' = a[1..ℓ]
        b' = a[2..ℓ + 1]
        i = 1
        while i ≤ ℓ do
            i++
        k = 1
        while k < 7 do
            p = StrangeAlgo4(a')
            a'[1] = p − 1
            k++
        i = 1
        while i ≤ n do
            j = 1
            while j ≤ n do
                j++
                b'[1] = min(a[i], a[j])
            i++
        return max(StrangeAlgo4(a'), StrangeAlgo4(b'))
```

Solution: Time: $T(n) = 8T(\lfloor n/7 \rfloor) + \Theta(n^2)$ (Note that the second while loop repeats only six times.)
Space: $M(n) = M(\lfloor n/7 \rfloor) + \Theta(n)$ (Note that the $\Theta(n)$ accounts for the memory of the current call and $M(\lfloor n/7 \rfloor)$ accounts for the total memory of the eight recursive calls to `StrangeAlgo4`—note that we didn't write $8 \cdot M(\lfloor n/7 \rfloor)$ as the eight recursive calls can reuse their memory. Why? Observe that we call them one after the other. Thus, if one recursive call ends, its auxiliary variables and arrays are discarded and their memory becomes available for the next recursive call. Consequently, we can count them as one call when computing the memory complexity.)

Note that we can simplify the above recurrences by writing $n/7$ instead of $\lfloor n/7 \rfloor$.

(b)

Input: An array $a[1..n]$ whose all elements are pairwise different and whose length n is a power of 2, i.e., $n = 2^d$ for some integer d.
Output: The array a is sorted increasingly.

```
SlowQuickSort(a[1..n])
    if n < 2 then
        return a
    pivot = 1
    a1[1..n][1..n] new empty two-dimensional array
    a2[1..n][1..n] new empty two-dimensional array
    p1 = 1
    p2 = 1
    while pivot ≤ n do
        for i = 1 to n do
            if a[i] < a[pivot] then
                a1[pivot][p1] = a[i]
                p1 = p1 + 1
            else if a[i] ≥ a[pivot] then
                a2[pivot][p2] = a[i]
                p2 = p2 + 1
        if p1 == n/2 + 1 and p2 == n/2 + 1 then
            break // Leaves the while loop immediately
        pivot++
        p1 = 1
        p2 = 1
    b1[1..p1−1] new empty array
    b2[1..p2−1] new empty array
    for i = 1 to n/2 do
        b1[i] = a1[pivot][i]
        b2[i] = a2[pivot][i]
    b1 = SlowQuickSort(b1)
    b2 = SlowQuickSort(b2)
    for i = 1 to n/2 do
        a[i] = b1[i]
        a[i + n/2] = b2[i]
    return a
```

Solution: Time: $T(n) = 2T(n/2) + \Theta(n^2)$
Space: $M(n) = T(n/2) + \Theta(n^2)$ (the two-dimensional array needs $\Theta(n^2)$ space, the two recursive calls count as one because the memory space of the first call can be reused by the second call.)

(c) What property has the element $a[pivot]$ immediately after the while loop is left?

Solution: It is the $((n/2) + 1)$ smallest element of a (or $(n/2)$-largest element of a). In other words, it is the larger one of the two middle elements (hence, it is a *median*).

(d) What happens, if we run `SlowQuickSort(`a`)` with an array a where all the elements are the same?

> **Solution:** The program never terminates as, in such a case, $b2$ is the same as a (since by assumption all elements are the same and in particular equal to $a[pivot]$, they are put into $a2$ and then to $b2$) and therefore `SlowQuickSort(`$b2$`)` is called forever.

Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 3.5: *Here and There*

You are given two arrays $here[1..n]$ and $there[1..n]$ containing numbers from $\{1, 2, 3\}$. There are n smileys numbered from 1 to n. For $1 \leq i \leq n$, $here[i]$ is the initial position of smiley i and $there[i]$ is the position it wishes to be. There are two rules:

1. All smileys on the same position are stacked over each other such that smileys with larger number are below smileys with smaller number.
2. A single step consists of taking one smiley from the top of one position and placing it on the top of some other position.

Write a function `fulfill_wishes` that writes how to fulfill the wishes of all the smileys (at the same time and without violating the rules at any point)! The output of your function should be a sequence of move operations of the form "Take the top-most smiley from position i and place it on top of position j".

> **Solution:** (Sketch) The task is a generalization of the known "Tower of Hanoi" problem. Write two auxiliary procedures: `from_here_to_k(`m`, `$here[1..n]$`, `k`)` and `from_k_to_there(`m`, `k`, `$there[1..n]$`)`. The first procedure moves the smileys $1, \ldots, m$ from their initial positions to the given position k. The second procedure moves the smileys $1, \ldots, m$ from the given position k to their desired positions. Then use both procedures to move all the smileys from *here* to *there*.

Algorithms for Data Science
Cheatsheet for Exercises 4

Howto: Backtracking → Memoization → Dynamic Programming

In the following, I explain in four steps on how to come up with a fast and space-efficient dynamic programming solution to many problems. At the end, I give some further notes.

Step 1: Backtracking Write a recursive algorithm using the divide and conquer technique (if possible).

```
MainAlgo(a)
    n = size of a
    return Algo(a, n)

Algo(a, k)                                          // k describes a subproblem of a
    if k is small then                                                  // Base case
        Compute the return value directly without recursion.
        return value
    else
        Make recursive calls on smaller values of k, combine the results, and compute the return value
        return value
```

Step 2: Memoization Store the return values to avoid recomputing them all over again.

1. Create a global empty dictionary within the main function: `mem = new dictionary` or `mem = Init()` (Pseudocode), `mem = {}` (Python)
2. Surround the code of step 1 with the following if query: `if mem.check(k)==false` (Pseudocode), `if k not in mem` (Pseudocode, Python)
3. Replace all return value statements with `mem.add(k, value)` (Pseudocode), `mem[k] = value` (Pseudocode, Python)
4. After the if block, return the appropriate value from the dictionary: `return mem.get(k)` (Pseudocode), `return mem[k]` (Pseudocode, Python)
5. That's it!

```
mem // a global variable
MainAlgo(a)
    mem = Init() // new dictionary
    n = size of a
    return Algo(a, n)

Algo(a, k)
    if k not in mem then                          // Same as: mem.check(k)==False
        if k is small then                                                  // Base case
            Compute the return value directly without recursion.
            mem[k] = value                        // Same as: mem.add(k, value)
        else
            Make recursive calls on smaller values of k, combine the results, and compute the return
             value
            mem[k] = value
    return mem[k]                                 // Same as: return mem.get(k)
```

Step 3: Dynamic Programming Make your solution from step 2 iterative going bottom to top by using dynamic programming.

1. Check what are the minimum and maximum possible values for k. Let us denote them by $k_{\min}$ and $k_{\max}$, respectively. (Typically, but not always, $k_{\min} = 0$ and $k_{\max} = n$.)
2. In the main function, change `mem = Init()` to `mem[`$k_{\min}$ `..` $k_{\max}$`] = new array filled with 0s`.
3. Afterwards, add a loop that calls the recursive function for all values of k in increasing order: `for k=`$k_{\min}$ `to` $k_{\max}$`: Algo(a, k)`
4. Change `return Algo(a, n)` to `return mem[`$k_{\max}$`]`.
5. In the recursive function, remove the if statement `if k not in mem` (but keep the code that was inside the if)
6. Replace all recursive calls of the form `Algo(`k'`)` by `mem[`k'`]`.
7. Replace everywhere `mem.add(k,value)` by `mem[k] = value`.
8. Remove `return mem.get(k)`.
9. Done! :-)

```
mem // a global variable
MainAlgo(a)
    mem[k_min .. k_max] = new array filled with 0s
    for k = k_min to k_max do
        Algo(a, k)
    return mem[k_max]

Algo(a, k)
    if k is small then                                          // Base case
        Compute the return value directly without recursion.
        mem[k] = value
    else
        Combine the results for smaller values of k stored in mem and compute the return value
        mem[k] = value
```

Step 4: Minimize Space You can substantially reduce the space if your computation of $mem[k]$ uses only some last few values of mem. In other words, there has to be some fixed number c such that the computation of $mem[k]$ does not depend on $mem[k_{\min} \,..\, k-c]$.

For example, if $mem[k] = mem[k-1]+mem[k-2]$, then its computation does not depend on $mem[0 \,..\, k-3]$). Hence, in that case we have $c = 3$.

Here's how it goes in three simple steps:

1. Check what is c in your algorithm.
2. Change the size of the array mem to c: `mem[0 .. c-1]`.
3. Everywhere change `mem[i]` to `mem[(i)%c]` (for any used value i) where % is the modulo operator.

That's it!

```
mem // a global variable
MainAlgo(a)
    mem[0 .. c−1] = new array filled with 0s
    for k = k_min to k_max do
        Algo(a, k)
    return mem[k_max%c]

Algo(a, k)
    if k is small then                                          // Base case
        Compute the return value directly without recursion.
        mem[k%c] = value
    else
        Combine the results for smaller values of k (modulo c) stored in mem and compute the
         return value
        mem[k%c] = value
```

Further notes:

- There are of course other ways to write memoization and dynamic programming code. Often, one can do it in a more compact way than in the general way presented above. For example, for the dynamic programming, we would use only one function in total without global variables and split the for loop in to two loops: one for the base case and the other one for the general case; see below.

```
MainAlgo(a)
    mem[0 .. c−1] = new array filled with 0s
    for all small k do
        Compute the return value directly without recursion.
        mem[k%c] = value
    for all big k do
        Combine the results for smaller values of k (modulo c) stored in mem and compute the
         return value
        mem[k%c] = value
    return mem[k_max%c]
```

- If the return value in the base case is the same for all small k, we can actually fill the array in the beginning with this value. Then we do not need the for loop for the base case anymore.

- Note that in the algorithms above, when writing `Algo(a,k)`, we pass only the memory address of the problem a (hence, a pointer to a), and not a copy of the problem a. In Python, this is done automatically: when you call function on a Python list (i.e., array), then actually only the pointer is passed and not the whole array copied.

  Our parameter k tells the function, which part of a is relevant; hence, it defines the subproblem.

- One could of course create a subcopy of the original problem a containing exactly the subproblem and pass this subproblem to the recursive call. Then the parameter k would not be needed anymore for the backtracking solution. However, such an approach is very bad: making a separate copy for every recursive call costs, in total, a lot of time and space. Thus, it is much more efficient as what we do above: to pass only a pointer to the original problem together with an integer parameter describing the relevant part for the suproblem.

  An efficient alternative to our approach is to declare the whole problem a as a global variable. Then only the parameter k has to be passed to the recursive calls. The asymptotic running time and space complexity remains the same.

- For some problems, we define their subproblems with more than one parameter, say t many parameters $k_1, \cdots k_t$. In such cases, you can treat k as a tuple of the form $(k_1, \ldots, k_t)$.

  For the base case, by writing `if k is small`, we mean that the values in the tuple are such that we can solve the subproblem directly. For example, it might be the case that we can solve the problem easily if k_1 is 0 independent of all the values of $k_2, \ldots, k_t$. Thus, in such a case, the if condition would only check only if k_1 is 0.

  The memoization step works perfectly fine for k being a tuple. The dynamic programming step, though, needs a modification: the array mem is now a t-dimensional array of the form $mem[k_{1\min} .. k_{1\max}] \ldots [k_{t\min} .. k_{t\max}]$. Instead of writing `mem[k]`, we now write `mem[`k_1`]...[`k_t`]`. The for loop `for k=` $k_{\min}$ `to` $k_{\max}$ has to be rewritten as t nested for loops, each one for a different parameter k_i from our tuple k. The order of the for loops might be crucial. Others than that, everything works fine.

Algorithms for Data Science
Exercise 4
(with Solutions)

See the cheatsheet for valuable remarks!

Task 4.1: *Dynamic Programming: Nontouching Cells (1D)*

You are given an array consisting of n non-negative values. Your task is to choose a subset of the array cells such that no two cells touch and the total value is maximized. It suffices that you output the total value.

2	1	1	2	0

An example array ($n = 5$). The chosen cells are shaded in gray. The total value is 4.

Input: array $val[1..n]$.
Output: total value of the chosen array cells.

Solve the problem via the following methods. What is the running time and memory complexity?

(a) Backtracking

Solution: The idea is as follows: starting with $i = n$, we want to compute the best solution to the array $val[1..i]$. To do so, we check both options

- taking the i-th element, and
- not taking the i-th element

and we decide for the option that maximizes the objective.
But how to compute the objective for each of the two options?

- The best solution for the case that we don't take the i-th element is the same as the the best solution to the subarray $val[1..i-1]$.
- The best solution for the case that we take the i-th element is the same as adding the value $val[i]$ of the i-th element to the best solution to the subarray $val[1..i-2]$.

Altogether, we obtain the following algorithm.

```
AlgoBack(val[1..n],i)
    if i == 0 then
        return 0
    if i == 1 then
        return val[1]
    return max(AlgoBack(val,i-1), val[i] + AlgoBack(val,i-2))
```

The running time recurrence is $T(i) = T(i-1) + T(i-2) + \Theta(1)$. (Note that this recurrence cannot be solved with the master theorem.) The recurrence is the same as for the Fibonacci example in the lecture. Thus, the resulting running time is $\Theta(\varphi^n)$, which is exponential.

Given that the maximum depth of recursion is n, the recursion stack and, hence, the used memory, occupies $\Theta(n)$ space.

Further note: if we write "$i \leq 0$" as the condition of the first if statement, then we do not need the second if statement at all. Thus, the code of our algorithm gets shorter and more elegant. I chose the variant above, as it will be later a little easier to transform it into a dynamic programming algorithm.

(b) Recursion with Memoization

Solution: We add a dictionary so that for each i, we compute the value only once.

```
memo = Init() // new dictionary
AlgoMem(val[1..n],i)
    if i == 0 then
        return 0
    if i == 1 then
        return val[1]
    if i not in memo then
        memo[i] = max(AlgoMem(val,i-1), val[i] + AlgoMem(val,i-2))
    return memo[i]
```

By implementing the dictionary via an array, we may assume that the access times to the dictionary are constant.

Thus, the total running time is in the order of the total number of recursive calls.

For each value of $i > 1$ there are two recursive calls but only the first time the function is called with this parameter. For subsequent calls or for values of $i \leq 1$, there are no recursive calls at all.

Thus, the total number of recursive calls (and hence the running time) is $\Theta(2(n-1)) = \Theta(n)$, which is linear in n.

Concerning the memory complexity, the recursion stack has the same size as in the backtracking solution, that is, size $\Theta(n)$. Additionally, we have to store our results for n different values of i in our dictionary. Thus, also our dictionary needs $\Theta(n)$ space. Hence, the total space usage is $\Theta(n)$.

(c) Dynamic Programming

Solution: Basically, we do the solution of memoization but bottom up and without recursion.

```
AlgoDP(val[1..n])
    memo[0..n] = new array
    memo[0] = 0
    memo[1] = val[1]
    for i = 2 to n do
        memo[i] = max(memo[i-1], val[i] + memo[i-2])
    return memo[n]
```

In contrast to the two recursive solutions, it is fairly easy to determine the running time of our solution based on dynamic programming. The time is dominated by the time to create the auxiliary array as well as to complete the main for loop. Both take $\Theta(n)$ time in our case, hence, our third solution takes linear time in total.

The space complexity is now determined only by the size of the array $memo[0..n]$, hence, we use $\Theta(n)$ space. If we could make the array somehow smaller, we could save memory! Can we? Yes, see the next subtask!

(d) Dynamic Programming (with only $O(1)$ extra space!)

Solution: At any moment, we only need to know the two last cells of our *memo* array. By using the modulo trick, we can thus reduce the memory to a constant! Changes with respect to the last subtask are in blue.

```
AlgoDP(val[1..n])
    memo[0..1] = new array
    memo[0] = 0
    memo[1] = val[1]
    for i = 2 to n do
        memo[i%2] = max(memo[(i − 1)%2], val[i] + memo[(i − 2)%2])
    return memo[n%2]
```

The running time does of course not change. Since modulo operations are in reality relatively expensive (but still cost only a constant number of basic arithmetic operations), it is more efficient in practice (but in theory we see no difference) to rewrite the algorithm using only two variables. (Below, the operation $x, y = e, f$ assigns at the same time e to x and f to y.)

```
AlgoDP(val[1..n])
    b = 0
    a = val[1]
    for i = 2 to n do
        a , b = max(a, val[i] + b) , a
    return a
```

(e) Modify the DP from subtask (c) such that it outputs the chosen grid cells (**Output:** binary array $chosen[1..n]$ where $chosen[i] = true$ if the grid cell in the i-th column has been chosen.)

Solution: Run the DP from subtask (c) to compute the DP table, and then go backwards through the array checking each time what was the best option: to take or not to take the i-th element.

```
AlgoDP(val[1..n])
    memo[0..n] = new array
    memo[0] = 0
    memo[1] = val[1]
    for i = 2 to n do
        memo[i] = max(memo[i − 1], val[i] + memo[i − 2])

    chosen[1..n] = new array filled with 0s.
    i = n
    while i > 1 do
        if memo[i] == val[i] + memo[i − 2] then
            chosen[i] = 1
            i = i − 2
        else
            i = i − 1
    if i == 1 then
        chosen[1] = 1
    return chosen
```

Task 4.2: *Dynamic Programming: Nontouching Cells (2D)*

Consider the following generalization of the last task (Task 4.1). You are given a $(k \times n)$-grid consisting of n columns and k rows. Each grid cell has some non-negative value. Your task is to choose **at most one** grid cell from each of the n columns such that no two grid cells are touching each other (touching with corners is also forbidden) and the total value is maximized. It suffices that you output the total value.

0	1	2	1	0
1	1	1	1	0
0	1	2	1	2
0	1	2	1	2

An example grid ($n = 5$ and $k = 4$). The chosen cells are shaded in gray. The total value is 6.

Input: array $val[1..n][1..k]$.

Output: total value of the chosen grid cells.

Solve the problem for any $k \geq 4$ via the following methods. What is the running time?

(a) Backtracking

Hint: First solve the problem for the restricted case where the input specifies some $j \in \{1, \ldots, k\}$ and, within the *last* column, we are allowed to pick only the grid cell of the j-th row.

Solution:

Running time: Exponential in n (in more detail: $k^{\Theta(n)}$) as for each column we have $2k$ recursive calls.

```
/* We call our function below trying all combinations for the last
   column.                                                          */
maxVal = 0
for j = 1 to k do
    maxVal = max(maxVal, AlgoBack(val,n,j))
return maxVal
```

```
/* The output is the maximum value attainable in val[1..i][1..k] under the
   condition that from the i-th column we may take only the
   element val[i][k].                                               */
AlgoBack(val[1..n][1..k], i, j)
    if i == 0 then
        return 0
    maxVal = 0
    // The case that we don't take val[i][j].
    for t = 1 to k do
        maxVal = max(maxVal, AlgoBack(val,i-1,t))
    // The case that we take val[i][j].
    for t = 1 to k do
        // The cell val[i-1][t] touches val[i][j] if t ∈ {j-1, j, j+1}.
           As k > 3, there is at least one non-touching cell.
        if t < j-1 or t > j+1 then
            maxVal = max(maxVal, val[i][j] + AlgoBack(val,i-1,t))
    return maxVal
```

(b) Recursion with Memoization

Solution: We just add a dictionary *memo* to the last solution. The modifications are in blue.

```
memo = Init()
AlgoMem(val[1..n][1..k], i, j)
    if i == 0 then
        return 0
    if (i, j) not in memo then
        maxVal = 0
        for t = 1 to k do
            maxVal = max(maxVal, AlgoMem(val,i-1,t))
        for t = 1 to k do
            if t < j − 1 or t > j + 1 then
                maxVal = max(maxVal, val[i][j] + AlgoMem(val,i-1,t))
        memo[i][j] = maxVal
    return memo[i][j]
```

For this problem, we can assume that the dictionary is a two-dimensional array $memo[1..n][1..k]$ (hence, we use the dictionary type "Array indexed by keys": initially, we fill the array with a value that never appears as a solution, for instance with -1. Thus, if we get the query "Is (i, j) **not in** *memo*?", we answer with true if and only if $memo[i][j] == -1$. Thus, the access and modification times for *memo* are constant in such a solution. The time to initialize such a dictionary is $\Theta(nk)$.

To analyze the total running time, there are two possibilities:

First possibility. Observe that the running time is asymptotically equal to the total number of recursive calls. To see it, your can argue as follows: charge the time of the for loops (where we make the recursive calls) to the functions that are recursively called. Consequently, the total time charged to each function call can be simplified to just $\Theta(1)$ (the time for the code besides the for loops—note that the time to access our dictionary is also $\Theta(1)$). Consequently, the total running time is just the number of recursive calls times $\Theta(1)$. Note that, within `AlgoMem(val,i,j)`, $\Theta(k)$ recursive calls are made but only if $i \geq 1$ and only for the first time `AlgoMem(val,i,j)` is run for the parameters i and j. For any subsequent runs (or if $i = 0$), there are no recursive calls at all within `AlgoMem(val,i,j)`. Therefore the total number of recursive calls is asymptotically equal to $\Theta(k)$ times the number of all combinations of $i \geq 1$ and k. Thus, the resulting running time (together with the time to initialize the dictionary) is $\Theta(nk^2)$.

Second possibility. The first time we call our function for a given combination of i and j, the running time is $\Theta(k)$ (the for loops) plus the time for the subsequent recursive calls (with the exception of $i = 0$, then the time is just $\Theta(1)$). For any subsequent calls of our function with the same combination of i and j, the running time is just equal to the time to access our dictionary, which is $\Theta(1)$ given the choice of our dictionary.
So, the total running time consists of three parts:

1. the total running time of first-time calls of our function,
2. the total running time of subsequent calls of our function, and
3. the time $\Theta(nk)$ to initialize our dictionary in the beginning.

The first part is just the number of all combinations of $i > 0$ and $j \geq 0$ times $\Theta(k)$ (plus $\Theta(1)$ times the number of combinations of $i = 0$ and $j \geq 0$). Hence, we get $\Theta(nk^2)$ for the first part in total.

The second part is not larger than the number of all recursive calls time $\Theta(1)$. In total, for each combination of i and j, we make at most k recursive calls. Thus, there are at most $O(nk^2)$ recursive calls. Hence, the running time of the second part is $O(nk^2)$ (in Oh-notation as we are lazy and argued only about the upper bound, which is enough as the first part takes already $\Theta(nk^2)$).

Altogether, we obtain $\Theta(nk^2)$ as the resulting running time.

(c) Dynamic Programming

Solution: We do everything bottom up. Time $\Theta(nk^2)$, Space $\Theta(nk)$.

```
DPtable[0..n][1..k] = new array filled with 0s
for i = 1 to n do
    for j = 1 to k do
        maxVal = 0
        for t = 1 to k do
            maxVal = max(maxVal, DPtable[i − 1][t])
        for t = 1 to k do
            if t < j − 1 or t > j + 1 then
                maxVal = max(maxVal, val[i][j] + DPtable[i − 1][t])
        DPtable[i][j] = maxVal
maxVal = 0
for t = 1 to k do
    maxVal = max(maxVal, DPtable[n][t])
return maxVal
```

(d) Dynamic Programming with $O(k)$ extra memory

Solution: Just record the current and the last column (in total two at any moment). Changes are in blue. If i is odd, than we use the column 1 in the DP table to store values for it, otherwise the column 0; hence, $i\%2$ gives us the column for i (where $i\%2$ means i modulo 2).

```
DPtable[0..1][1..k] = new array filled with 0s
for i = 1 to n do
    for j = 1 to k do
        maxVal = 0
        for t = 1 to k do
            maxVal = max(maxVal, DPtable[(i − 1)%2][t])
        for t = 1 to k do
            if t < j − 1 or t > j + 1 then
                maxVal = max(maxVal, val[i][j] + DPtable[(i − 1)%2][t])
        DPtable[i%2][j] = maxVal
maxVal = 0
for t = 1 to k do
    maxVal = max(maxVal, DPtable[n%2][t])
return maxVal
```

(e) (Bonus Task) Modify the algorithm of task (c) such that it outputs the chosen grid cells as a binary array $chosen[1..n][1..k]$ where $chosen[i][k] = true$ if the grid cell in the i-th column and j-th row has been chosen.

Solution: When finished computing the DP table go back through the table and write into *chosen* which element you choose.

Input: $maxVal$ and $DPtable[0..n][1..k]$ as computed by our dynamic program.
Output: $chosen[1..n][1..k]$

```
// To make our algorithm simpler, we assume that we have a dummy element
   in the non-existing column n+1 at the non-existing row k+2 (that
   does not touch any other cells of the actual input table). Starting
   from this dummy element, the current candidate has coordinates (i,j)
   and we want to find out whether to add it or not.
```

$chosen[1..n+1][1..k]$ = new boolean array; initially all entries set to false; `// for technical reasons` $n+1$ `columns`
$i = n+1$
$j = k+2$
$valij = 0$`// Later, when` $i \leq n$`, it will be equal to` $val[i][j]$.
while $i \geq 1$ **do**
  $notFound = true$ `// Is set to` $true$ `as long as we don't know from which cell and in what case we reached` (i,j)

  `// Case 1: We don't take element` (i,j)
  $t = 1$
  while $t \leq k$ **and** $notFound$ **do**
    if $maxVal == DPtable[i-1][t]$ **then**
      $notFound = false$
      $j = t$
    $t = t+1$

  `// Case 2: We take element` (i,j)`. This case must hold if` $notFound$ `is still set to` $false$.
  $t = 1$
  while $notFound$ **do**
    if $(t < j-1$ **or** $t > j+1)$ **and** $maxVal == valij + DPtable[i-1][t]$ **then**
      $notFound = false$
      $chosen[i][j] = true$ `// Note that the` $j = k+2$ `value of the first candidate would lead to an error as` $chosen$ `has only size` $[1..n+1][1..k]$. `However, Case 1 is always satisfied for the first candidate; thus, we will never reach this line of code with the first candidate.`
      $j = t$
    $t = t+1$

  `// The next candidate is at column` $i = i-1$ `and at row` j `(we computed` j `in Case 1 or 2)`
  $i = i-1$
  $valij = val[i][j]$
  $maxVal = DPtable[i][j]$

Remove the last (technical) column from $chosen$
`// Now,` $chosen$ `has the same size as` val`, that is, size` $[1..n][1..k]$.
return $chosen$

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 4.3: *Dynamic Programming: Maximum Block*

Given an array containing possibly negative values, compute the value of a block of maximum total value.

Formally, let $a[1..n]$ be the input array. The problem is to output the total value of a *heaviest* block, that is, of a subarray $a[i..j]$ of maximum total value $\sum_{k=i}^{j} a[k]$.

Solution: ***The solutions are actually very simple (see any of the final algorithms below). In the following text, I give you some intuition on how to come up with the final solution in two different ways.***

Solution 1 (Start with Recursion): Before you try recursion, you might try the very direct approach of just computing

$$\max_{1 \le i \le j \le n} \sum_{k=i}^{j} a[k] \ .$$

```
AlgoSlow(a[1..n])
    max_sum = 0
    for i = 1 to n do
        for j = i to n do
            sum = 0
            for k = i to j do
                sum = sum + a[k]
            if sum > max_sum then
                max_sum = sum
    return max_sum
```

The time is $\Theta(n^3)$ and space complexity is $\Theta(1)$. Space complexity is optimal and the time is polynomial. Thus, the time is not so bad, but is it possible to do it better? If we think for short while, we notice, that in the inner most loop we recompute some parts over and over again. After some thinking, we might end up with the following $\Theta(n^2)$-time algorithm that again has optimal $\Theta(1)$ space complexity.

```
AlgoFaster(a[1..n])
    max_sum = 0
    for i = 1 to n do
        sum = 0
        for j = i to n do
            sum = sum + a[j]
            if sum > max_sum then
                max_sum = sum
    return max_sum
```

But is this quadratic running time optimal?

Let's try a recursive approach using divide and conquer! In the direct approach above, *max_sum* was the best solution found so far in the subarray $a[1..i]$. So, let's also use such a parameter i. If i is 0, we define the array to be empty and return 0. Otherwise, we reduce the problem to a smaller one. We have one decision to make: to take or not to take $a[i]$ into our block. If we take it, then we either have also to take the next element, $a[i-1]$, or to stop the recursion (meaning, that $a[i]$ begins the block) as the block has to consist of consecutive elements. Thus, we will have also a second boolean parameter *take_or_stop*.

- If it is **False**, it means we haven't started a block yet, and we can decide whether we skip the i-the element, or whether we take it and start a block. In our algorithm, we check both cases recursively and return the maximum.
- If *take_or_stop* is **True**, we have to either stop the recursion or take the i-th element. Again, we check both cases and take the maximum of 0 (stopping the recursion) and the recursively obtained value when choosing $a[i]$.

We call the function with the parameters $i = n$ and *take_or_stop* = **False**.

```
AlgoRec(a[1..n], i, take_or_stop)
    if i == 0 then
        return 0;
    else
        if take_or_stop then // if take_or_stop is True
            return max(0, a[i] + AlgoRec(a,i-1,True))
        else
            return max(AlgoRec(a,i-1,False), a[i] + AlgoRec(a,i-1,True))
```

Surprisingly, the running time is not exponential as in many other recursive approaches, but $\Theta(n^2)$—thus as good as our best direct approach above! To see the quadratic running time, take a sheet of paper and draw a node for each combination of the two parameters. Draw an arrow (starting nowhere) to the node with $i = n$ and *take_or_stop* = **False**. Next, if some node A has t incoming arrows, draw t arrows from A to each node that is called by A. The total number of arrows is now asymptotic to the total running time. You will notice that nodes with the parameter *take_or_stop* = **False** have exactly one incoming arrow whereas nodes with *take_or_stop* = **True** have exactly $n - i$ incoming arrows. Thus, the number of arrows is

$$\Theta(n + \sum_{i=0}^{n}(n-i)) = \Theta(n + \sum_{i=0}^{n} i) = \Theta(n + n^2) = \Theta(n^2) \ .$$

Since the running time of the recursive approach is already so good, will it become even better if we add memoization?

```
mem = Init() // new dictionary
AlgoMem(a[1..n], i, take_or_stop)
    key = (i, take_or_stop)
    if key not in mem then
        if i == 0 then
            mem[key] =0
        else
            if take_or_stop then // if take_or_stop is True
                mem[key] = max(0, a[i] + AlgoMem(a,i-1,True))
            else
                mem[key] = max(AlgoMem(a,i-1,False), a[i] + AlgoMem(a,i-1,True))
    return mem[key]
```

Now, what is the running time? If we take a picture with the nodes that we did above, we notice that suddenly each node sends exactly one arrow (for $i > 0$). Thus, the total number of arrows is the number of nodes which is the number of combinations of i and true/false values, which is $\Theta(n)$. And this is our running time! Nice, we achieved linear running time using recursion with memoization. So is this algorithm better than the direct approach above? In terms of time: yes, definitely! In terms of memory: no! Our algorithm with memoization uses $\Theta(n)$ extra space which is much worse when compared to the constant space used by our direct approach above.

Is there any hope to use less space? Yes, by formulating a dynamic programming algorithm and then trying to reduce the space.

By going the recursive approach backwards from bottom ($i = 0$) to top ($i = n$), we get the following dynamic programming solution using a two-dimensional array $dp[0..n][0..1]$. Our goal is to directly compute dp such that that $d[i][b]$ equals $mem[(i, b)]$ (where we interpret $b = 0$ as **False** and $b = 1$ as **True**).

```
AlgoDP(a[1..n])
    dp[0..n][0..1] new array filled with 0s. for i = 1 to n do
        dp[i][1] = max(0, a[i] + dp[i − 1][1]
        dp[i][0] = max(dp[i − 1][0], a[i] + dp[i − 1][1]
    return dp[n][0]
```

It's easy to see that time time and space is $\Theta(n)$. Can we reduce the space now? Observe that $dp[i][0]$ and $dp[i][1]$ depend only on position $i - 1$ of the DP array. Hence, using the modulo trick, we can shrink the DP table to $dp[0..1][0..1]$ and thus obtain $\Theta(1)$ space.

```
AlgoDP(a[1..n])
    dp[0..1][0..1] new array filled with 0s. for i = 1 to n do
        dp[i%2][1] = max(0, a[i] + dp[(i − 1)%2][1]
        dp[i%2][0] = max(dp[(i − 1)%2][0], a[i] + dp[(i − 1)%2][1]
    return dp[n%2][0]
```

Note that we actually do not need to keep $dp[i\%2][0]$ and $dp[(i - 1)\%2][0]$ at the same time in the memory. By rewriting the code a little bit by using an auxiliary variable *temp*, we neither need to

keep $dp[(i-1)\%2]$. Thus, we can shrink the table more do $dp[0..0][0..1]$, meaning that it becomes a one-dimensional table $dp[0..1]$ independent of i.

```
AlgoDP(a[1..n])
    dp[0..1] new array filled with 0s. for i = 1 to n do
        temp = a[i] + dp[1]
        dp[1] = max(0, temp)
        dp[0] = max(dp[0], temp)
    return dp[0]
```

Instead of using the table, it's more efficient to replace it by two variables. We can even get rid of the *temp* variable, optimizing the code down to two variables in total (not counting the iterator i).

```
AlgoDP(a[1..n])
    c = 0, d = 0
    for i = 1 to n do
        d = a[i] + d
        c = max(c, d)
        d = max(0, d)
    return c
```

Note that we can further shorten the code by taking the maximum with 0 already in the first line of the loop.

```
AlgoDP(a[1..n])
    c = 0, d = 0
    for i = 1 to n do
        d = max(0, a[i] + d)
        c = max(c, d)
    return c
```

Thus, starting with the recursive approach, we ended up with a very efficient and elegant iterative approach (with time $\Theta(n)$ and space $\Theta(1)$).

Solution 2 (Start directly with DP): How to solve such a problem using recursion or dynamic programming? Let us take a look at what choices we have:

- If we choose to take some element $a[j]$ into the block, then (i) either $a[j]$ is the first element of the block—meaning that we cannot take any element $a[k]$ with $k < j$ to the block—, or (ii) $a[j]$ is some middle element meaning that we have to choose $a[j-1]$ also to the block and to again consider the two options (i) and (ii) for the element $a[j-1]$ (hence, we have a recursion here!).

  By this reasoning we can compute the maximum total value of a block within the subarray $a[1..j]$ assuming that the block ends in $a[j]$; let us call this value $DPtake[j]$. In case (i), the block also starts in $a[j]$, hence, consists only of $a[j]$ and has therefore value just $a[j]$. In case (ii), the block consists of $a[j]$ and—this is now the important observation—of a heaviest block that ends in $a[j-1]$, that is, of value $DPtake[j-1]$. Hence, to compute $DPtake[j]$, we have to take the maximum over both cases (i) and (ii):

  $$DPtake[j] = \max(a[j], a[j] + DPtake[j-1]) \ .$$

  Now, the question arises: Couldn't we just write $DPtake[j] = a[j] + DPtake[j-1]$? The answer is no as $DPtake[j-1]$ might be negative—recall that we have negative numbers!

- OK, but what if we don't choose to take the element $a[j]$ into the block? Then we have to find a block to the left and a block to the right of j of total maximum value. Let $DP[k]$ be the value of a heaviest block in the subarray $a[1..k]$ independently of whether $a[k]$ has been taken). Then the heaviest block to the left of j has value $DP[j-1]$. Let us skip the right side for now and ask, how to compute $DP[j]$.

Above, we have seen two choices, to take or not to take $a[j]$. What is the better choice? If we are interested in a heaviest block in the subarray $a[1..j]$ (whose value is defined to be $DP[j]$), then the answer is to take the maximum over both choices:

$$DP[j] = \max(DPtake[j], DP[j-1]) \ .$$

But what about the right side, that is, the subarray $a[j+1..n]$? Well, at the end of the day, we are interested in computing $DP[n]$ which is in fact the answer to the overall problem: the total value of a heaviest block in $a[1..n]$! Therefore, if we are able to compute $DP[n]$ without considering the right sides for particular values for j, we are totally fine!

Since $DP[j]$ depends only on $DP[j–1]$ and $DPtake[j]$, whereas $DPtake[j]$ depends only on $DPtake[j–1]$ and $a[j]$, we can compute $DP[n]$ by computing the respective values from left to right (for increasing j) or in a recursive manner.

Our discussion leads directly to the following dynamic programming algorithm:

```
Block_DP(a[1..n])
    DP[0..n], DPtake[0..n] = new arrays filled with zeros
    for j = 1 to n do
        DPtake[j] = max(a[j], a[j] + DPtake[j − 1])
        DP[j] = max(DPtake[j], DP[j − 1])
    return DP[n]
```

Since the for loop repeats only n times and has only operations that take constant time, the total running time is $\Theta(n)$. The space complexity is $\Theta(n)$ due to the two arrays DP and $DPtake$. However, we can even reduce the space complexity to $\Theta(1)$ by observing that for computing the values for index position j, we only need the index value of position $j-1$. Thus, arrays of size two suffice:

```
Block_DP_lessspace(a[1..n])
    DP[0..1], DPtake[0..1] = new arrays filled with zeros
    for j = 1 to n do
        DPtake[j%2] = max(a[j], a[j] + DPtake[(j − 1)%2])
        DP[j%2] = max(DPtake[j%2], DP[(j − 1)%2])
    return DP[n%2]
```

If we look more closely, then we even notice that we can discard the values for position $j-1$ in in the moment we finish computing the values for position j. Hence, we do not need an array at all, but two variables c and d suffice:

```
Block_DP_lessspace(a[1..n])
    c = 0, d = 0
    for j = 1 to n do
        d = max(a[j], a[j] + d)
        c = max(d, c)
    return c
```

To make the picture complete, we also discuss the recursive solutions. We begin by solving the problem recursively via backtracking. We have one method corresponding to DP and another corresponding to $DPtake$. We call the algorithm via `Block_Back(a,n)`.

```
Block_Back(a[1..n],j)
    if j == 0 then
        return 0
    return max(Block_Back(a,j-1), Blocktake_Back(a,j))
Blocktake_Back(a[1..n],j)
    if j == 0 then
        return 0
    return max(a[j], a[j] + Blocktake_Back(a,j-1))
```

The algorithm above has a high running time but—interestingly—it is not too horrible. Observe that `Block_Back(a,j)` is called exactly once for every value of j (and has constant time per call—not counting the time within the recursive calls). Thus, the total running time alone for all calls

of `Block_Back()` is $\Theta(n)$. What about `Blocktake_Back(a,j)`? For each j, it is called $n-j+1$ times: once by `Block_Back(a,j)` and $n-j$ times by `Blocktake_Back(a,j+1)`. The total running time is therefore $\Theta(\sum_{j=1}^{n}(n-j+1))$ which can be shown to be $\Theta(n^2)$. Hence, not so bad.

But, we can do it better using memoization and improve the running time down to $\Theta(n)$. We use two dictionaries, called DP and $DPtake$.

```
DP = Init(), DPtake = Init()
Block_Mem(a[1..n],j)
    if j == 0 then
        return 0
    if j not in DP then
        DP[j] = max(Block_Mem(a,j-1), Blocktake_Mem(a,j))
    return DP[j]
Blocktake_Mem(a[1..n],j)
    if j == 0 then
        return 0
    if j not in DPtake then
        DPtake[j] = max(a[j], a[j] + Blocktake_Mem(a,j-1))
    return DPtake[j]
```

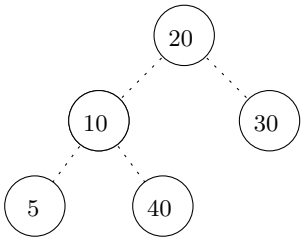
Task 4.4: *Heap Sort*

Run heap sort on the array $a = \langle 20, 10, 30, 5, 40\rangle$. Draw the array a after each single change as a binary complete tree.

Solution: First, the array is transformed into a heap. Here, we do it by adding the elements one after the other. One could also call the method `BuildHeap()` from the literature which would result in a different heap tree. After obtained the heap, we remove the maximum element until the heap is empty.

For clarity, changes are marked in yellow and edges that do not belong to the heap are dashed.

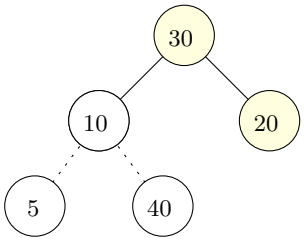
Beginning:



(1)

No change after `Add(20)` and `Add(10)`

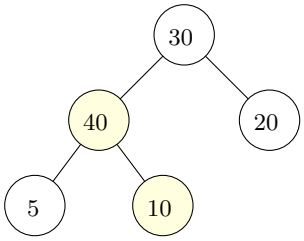
Change after `Add(30)`



(2)

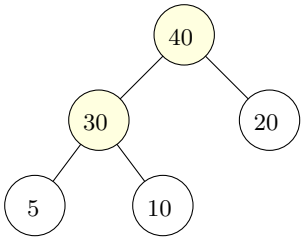
No change after `Add(5)`

First change during `Add(40)`



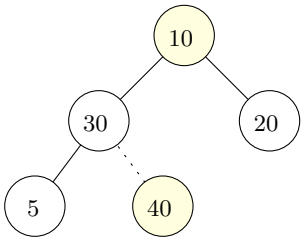
(3)

Secong (last) change during `Add(40)`



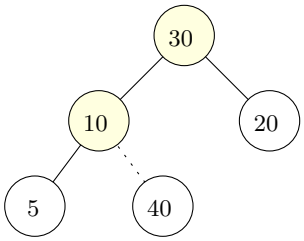
(4)

First change during first `RemoveMax()`



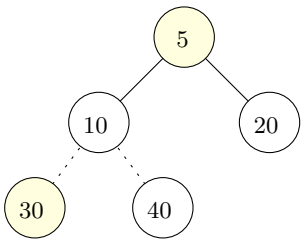
(5)

Second (last) change during first `RemoveMax()`



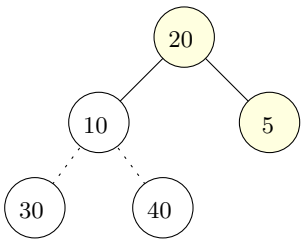
(6)

First change during second `RemoveMax()`



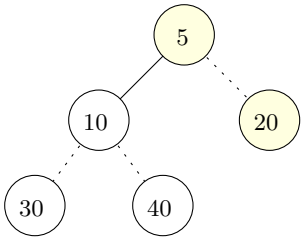
(7)

Second (last) change during second `RemoveMax()`



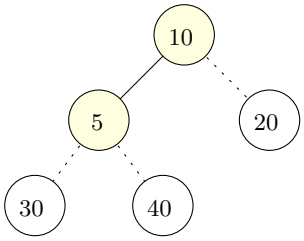
(8)

First change during third `RemoveMax()`



(9)

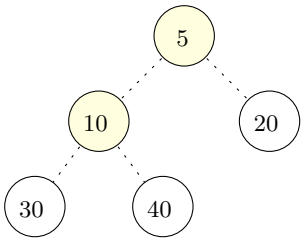
Second (last) change during third `RemoveMax()`



(10)

Change after forth `RemoveMax()`

(No change after fifth `RemoveMax()`)



(11)

Task 4.5: *Quick Sort: Polish Flag*

Second May is Polish National Flag Day. For this occasion, you want to program a robot to arrange red and white balls to form a Polish flag. In particular, you are given $2n$ bins numbered from 1 to $2n$ where each bin contains exactly one ball. In total there are n red and n white balls. Your goal is that the first n bins contain red balls, the remaining ones white balls. Your robot can only do the following operations:

- isRed(i) : the robot looks at the i-th bin and returns true if it contains a red ball, and false otherwise.
- swap(i,j): the robot exchanges the balls of the i-th and j-th bin.

Furthermore, your robot has only a small memory of constant size.

(a) How is this problem related to quick sort?

Solution: The Polish flag problem is equivalent to the subproblem of quick sort where elements smaller than the pivot (let's call them red) have to be moved to the left side of the pivot and elements larger than the pivot (let's call them white) have to be moved to the right side of the pivot. For instance, if we color our elements like this, then any algorithm for the Polish flag problem will move the elements correctly. At the end, we just have to reinsert the pivot element in between them. On the other hand, the quick sort subroutine would solve the Polish flag problem directly if we define the order *red is smaller than white* (or, alternatively, give all red balls the value 0 and all white balls the value 1).

Hence, if we can solve the Polish flag problem using only *a small memory of constant size* as required, then the quick sort algorithm for the subproblem, and in fact, for the overall sorting

problem, can also be implemented by using only constant additional memory. In the next subtask, we observe that this is in fact possible.

(b) Program the robot to arrange a Polish flag within $O(n)$ operations.

Solution:

```
left = 1
right = n
// The invariant: Anything to the left of left is red, and anything to
   the right of right is right.
while left < right do
    // The exclamation mark in !isRed(left) negates the output. That is,
       the following if statement should be read as:
    // If isRed(left) == false and isRed(right) == true
    if !isRed(left) and isRed(right) then
        swap(left,right)
    if isRed(left) then
        left = left + 1
    if !isRed(right) then
        right = right - 1
```

The algorithm terminates as, in each repetition of the while loop, we increase $left$ or decrease $right$. Thus, we repeat the for loop at most $n-1$ times and therefore the running time is linear in n.

The invariant is correct as we increase or decrease $left$ and $right$ only if they are red or white, respectively. Consequently, the output is correct.

Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 4.6: *Fibonacci Numbers*

We define the Fibonacci numbers as follows:

$$F_n = \begin{cases} n & n = 0, 1 \\ F_{n-1} + F_{n-2} & n > 1 \end{cases}$$

(a) How many times do we perform an addition if compute F_n recursively with the formula above?

Solution: Asymptotically, there are $\Theta(F_n)$ additions because if we resolve the recurrence, we get a sequence of zeros and ones with plus symbols in between. Obviously, the number of ones is exactly F_n, whereas the number of zeros is at most F_n as each zero appears in a sum with a one. Thus, there are at least $F_n - 1$ and at most $2F_n - 1$ additions.

The exact number of additions is $F_{n+1} - 1$. One can prove this by mathematical induction.

(b) Give an algorithm that computes F_n using only $O(\log n)$ arithmetic operations. Use the following recurrence:

$$\text{For } n > 1, \quad F_{2n-1} = F_n^2 + F_{n-1}^2 \quad \text{and} \quad F_{2n} = F_n^2 + 2F_nF_{n-1} \ .$$

Solution: Observation: The given recurrence implies that all the three numbers F_{n-1}, F_n and F_{n+1} can be computed by using only the three numbers $F_{\lfloor n/2 \rfloor -1}$, $F_{\lfloor n/2 \rfloor}$ and $F_{\lfloor n/2 \rfloor +1}$. Thus, one can design a recursive algorithm that on input n computes these three values by doing a single recursive call on $\lfloor n/2 \rfloor$. After a logarithmic number of calls, we reach the base case $n = 0, 1$.

(c) Give an algorithm that computes F_n using only $O(\log n)$ arithmetic operations. Use the following matrix recurrence:

$$\text{For } n > 1, \quad \begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} F_{n-1} \\ F_{n-2} \end{bmatrix} \ .$$

Solution: The recurrence implies the following equation:

$$\text{For } n > 1, \quad \begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \begin{bmatrix} F_1 \\ F_0 \end{bmatrix} \ .$$

Thus, the problem boils down to compute the matrix $\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1}$ using $O(\log n)$ arithmetic operations (addition and multiplication). This is can be done recursively using the following trivial recurrence for a matrix M:

$$M^{2n} = M^n M^n \qquad \text{and} \qquad M^{2n+1} = MM^nM^n \ .$$

Note: In the RAM model, each basic arithmetic operation costs $O(1)$ time. This is realistic as long as the values are not too large. However, in the Fibonacci algorithms above such an assumption is not realistic as the n-th Fibonacci number has $\Theta(n)$ bits in its standard encoding, and consequently it takes $\Theta(n)$ time just to write it down, not to mention the time to compute it. Thus, to be more realistic, we should assume that the addition of two n-bit numbers takes $\Theta(n)$ time, whereas multiplication takes $O(n \log n)$ time. (It is an open question in computer science whether one can do the multiplication faster. The best-known $O(n \log n)$ algorithm for multiplication is relatively young from 2019.) Summarized, the best-known algorithm to compute the n-Fibonacci number takes $O(n \log^2 n)$ time.

Algorithms for Data Science
Cheatsheet for Exercises 5

In a dictionary, each data entry that we store consists of two components: a *key* (often a number) and a value (any data type). The entries are therefore called *key-value pairs*.

Example: key = sudent's ID, value = student's name and address.

Invariant (Dictionary): *Each key appears at most once in a dictionary.*

Note that there are no such restrictions on the values. The same value can appear with different keys.

The method `Add(k,v)` fist checks whether the dictionary contains an entry with the key equal to k. If it contains, the old value is replaced by the new value v. If it does not contain such an entry, a new entry is stored: with the key k and the value v.

The dictionary **hash table** is an array together with a hash function that, given a key, computes the position in the array that is responsible for this key.
Hash function have often the following form: $h(k) = k \mod p$, where p is the size of the array and where mod is the modulo operator.

Definition of Modulo: If $k \geq 0$ and $p > 0$ are integers, then "k modulo p" (alternative writing: $k \mod p$ and $k\%p$) is

- the remainder of the division k/p, formally:
- $k - p \cdot \lfloor k/p \rfloor$.

A simple algorithm for computing $k \mod p$:

```
Modulo(k, p)
    while k ≥ p do
        k = k − p
    return k
```

The dictionary **AVL tree** is a **balanced binary search tree**.

Definitions:

- A tree is *binary* if each node has at most two children (called left and right).
- A binary tree is a binary search tree (BST) if each node has a key and satisfies the BST invariant (see below).
- A tree is balanced if its height is $O(\log n)$ where n is the number of nodes.
- The *height of a tree* is the height of its root node.
- The *height of a node* v is the length of a longest path starting in v and going always to the bottom until it reaches a node that has no children. The length of the path is measured by the number of nodes that we visit including v. Thus, if v has not children, then its height is 1.
- An AVL tree is a balanced BST where each node satisfies the AVL tree invariant (see below).

In the exam, short tests or exercise sessions, when you are asked for an algorithm for a binary search tree, you can make the following assumptions:

- We have the variable type `node` that corresponds to a node in a binary search tree.
- In most algorithms, you will start with the node called *root* that corresponds to the root of the tree.
- If the tree is empty, then `root==NONE` (meaning that the root is not set up yet).
- If v is a node (and `v` $\neq$ `NONE`), then it has the following attributes:
  - `v.key` is the key stored in v,
  - `v.value` is the value stored in v,
  - `v.left` is a pointer to the left child of v (set to `NONE` if it does not have a left child),
  - `v.right` is a pointer to the right child of v (set to `NONE` if it does not have a right child), and
  - `v.parent` is a pointer to the parent of v. If v is the root of the tree, then its parent is set to `NONE`.

  Here an example of a sequence of commands creating a node with key 5 and value 123456 that has a right child whose key is 6 and whose value is unspecified:

```
node v = new node
v.key = 5
v.value = 123456
v.left = None
w = new node
w.key = 6
v.right = w
w.parent = v
```

Binary Search Tree (BST) invariant: *For every node v, all keys in its left subtree are strictly less than the key of v, and all keys in its right subtree are strictly greater than the key of v.*

AVL tree invariant: *For every node v with at least one child: If v has only one child, then its child has height* 1. *If v has two children, then the heights of the children of v differ by at most* 1.

A node v without a left or a right child has `v.left==NONE` or `v.right==NONE`, respectively. If we define "NONE" to be a virtual/dummy node of height 0, then we can simplify the AVL tree invariant as follows.

AVL tree invariant (simplified): *For every node v, the heights of the children of v differ by at most* 1.

A **Heap** is not a dictionary and **not** a binary search tree! However, it is a complete binary tree. *Complete* means that the tree looks identical to tree obtained by filling the levels (nodes of the same height) one by one from top to bottom, and each level from left to right. A heap can store just values, or value-priority pairs. In the former case, the heap invariant is with respect to the values, in the latter case, with respect to the priorities.

Heap invariant: *For every node v, its priority is not smaller than the priority of its children!*

Thus, a heap may contain multiple value-priority pairs with the same priority!

Algorithms for Data Science
Exercise 5
(with Solutions)

See the cheatsheet for valuable remarks!

For simplicity, in some of the tasks we consider only the keys of the key-value pairs.

Task 5.1: *Simple Tasks: Hash and BST*

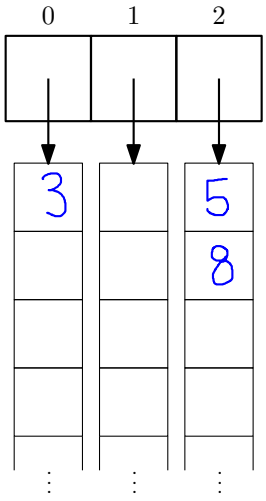
(a) Add the following keys in the given order into a hash table of size 3 using the hash function

$$h(k) = k \mod 3$$

keys: $3, 5, 8$

Conflicts are resolved via *separate chaining* as in the lecture, that is, each hash table cell stores a pointer to an unordered array where the elements are actually stored.

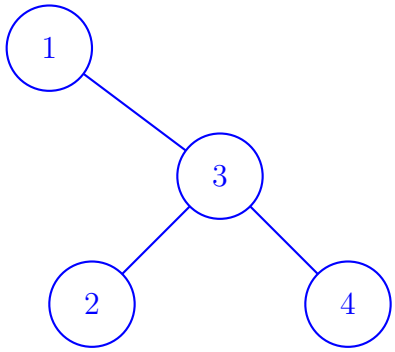
Solution:



(b) Add the following keys in the given order to an initially empty binary search tree:

$$1, 3, 4, 2$$

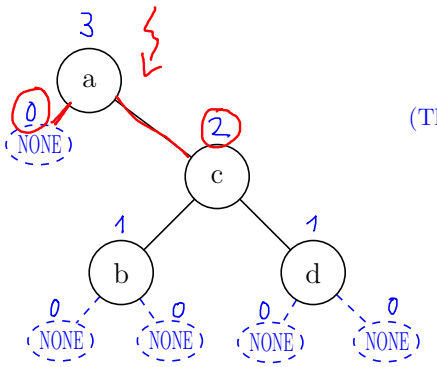
Solution:



(c) Compute the height for each node and check whether the AVL-invariant is satisfied. *Recall that the height of a node v is the number of* nodes *on a longest path starting at v within the subtree of v.*

Solution:

(The **NONE**-nodes are not real nodes. They just symbolize that the respective child does not exist. They do not count to the height of real nodes. Their height is 0.)



(The numbers are the heights)

(d) Design an algorithm `FindMin(`*root*`)` in pseudocode that returns the minimum key of the binary search tree rooted at *root* (whose root node is *root*). You can assume that the tree contains at least one element.

Solution:

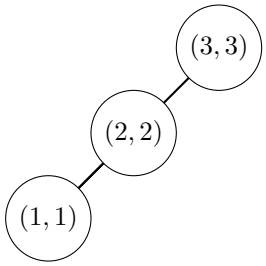
```
findMinKey(root)
    node = root
    while node.left ≠ NONE do
        node = node.left
    return node.key
```

(e) Draw a binary search tree that additionally satisfies the heap property with respect to the priorities containing the following key-priority pairs:

$$(1,1),(2,2),(3,3)$$

The tree does not need to be a complete binary tree as required by our definition of a heap. It just have to be a binary tree that satisfies the BST invariant and heap invariant at the same time.

Solution:



Task 5.2: *Hash Tables*

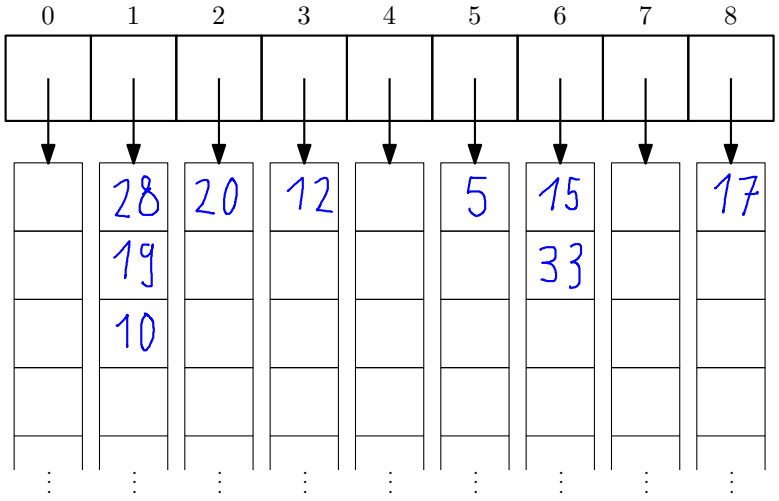
Add the following keys in the given order into a hash table of size 9 using the hash function

$$h(k) = k \mod 9 \ .$$

keys: $5, 28, 19, 15, 20, 33, 12, 17, 10$

Conflicts are resolved via *separate chaining* as in the lecture, that is, each hash table cell stores a pointer to an unordered array where the elements are actually stored.

Solution:

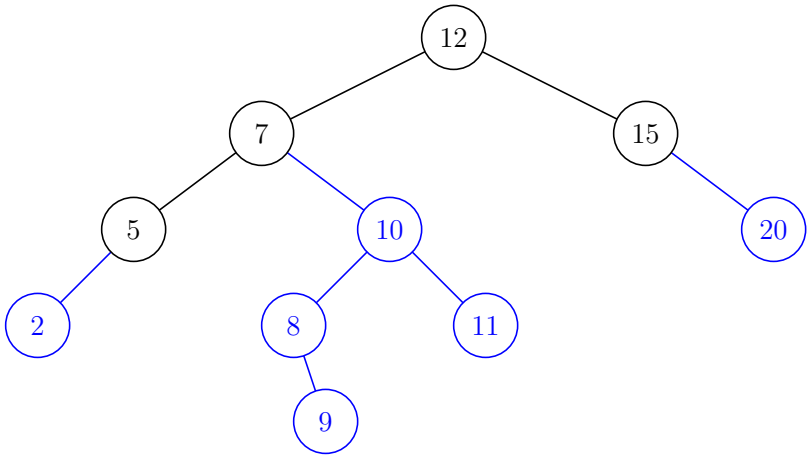


Task 5.3: *Binary Search Trees*

(a) Given the binary search tree below, add the following keys in the given order:

$$10, 20, 8, 5, 2, 9, 11$$

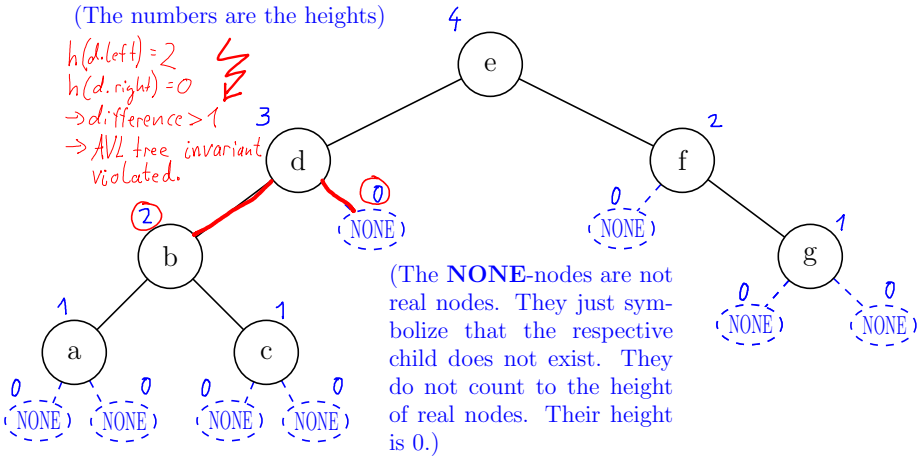
Solution:



Note that the key 5 is already in the tree, so the tree does not change (and only the associated value would be updated).

(b) Compute the height for each node and check whether the AVL-invariant is satisfied.

Solution:



(c) Design an algorithm `Add`($root$, k, v) in pseudocode that adds the key-value pair (k, v) to the binary search tree rooted at $root$. (If the tree is empty, $root$ = NONE.) The added node should be returned.

Solution: We will use the following auxiliary function:

```
CreateNewNode(k,v)
    w = new node
    w.key = k
    w.value = v
    w.left = NONE
    w.right = NONE
    return w
```

One possible solution using recursion:

```
Add(root,k,v)
    return AddRec(root, false, NONE, k, v)
AddRec(node, isLeftChild, parent, k, v)
    if node == NONE then
        node = CreateNewNode(k,v)
        if parent ≠ NONE then
            if isLeftChild then
                parent.left = node
            else
                parent.right = node
    return node
    else
        if k < node.key then
            return AddRec(node.left, true, node, k, v)
        else if node.key > k then
            return AddRec(node.right, false, node, k, v)
        else
            node.value = v
            return node
```

And another recursive solution, without extra parameters:

```
Add(root,k,v)
    if root == NONE then
        root = CreateNewNode(k,v)
        return root
    else
        return AddRec(root, k, v)
AddRec(root,k,v)
    if k < root.key then
        if root.left == NONE then
            w = CreateNewNode(k,v)
            w.parent = root
            root.left == w
        else
            w = AddRec(root.left, k, v)
        return w
    else if root.key < k then
        if root.right == NONE then
            w = CreateNewNode(k,v)
            w.parent = root
            root.right == w
        else
            w = AddRec(root.right, k, v)
    else
        root.value = v
        w = root
    return w
```

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 5.4: *Which Data Structure?*

For each of the following scenarios, propose the most appropriate data structure from our lecture (in terms of memory and time complexity). Explain your choice.

(a) You want to keep track of all winning lottery numbers of a special lottery where the numbers have an unrealistic huge number of digits, say, several billions. You want to store each number together with its date, and you also want to check whether a given number has won in the past. The operations should be as fast as possible in practice and the memory usage not too large.

Solution: Correct answer: Hash table.
Explanation: The lottery numbers are random and therefore the average access time is constant with high probability; hence, very fast in practice. (Note that by randomly picking a hash

function from a universal hash family, we could keep the time constant—with high probability—also in the case of non-random lottery numbers.) Othe data structures are worse. Arrays indexed by keys take too much memory and are therefore infeasible. Infeasible are also find-union, stack, queue, and heap as they do not provide operations to check if a a specific number has been added to them. The remaining data structures covered in the lecture (AVL tees, linked lists, sorted/unsorted arrays) have much longer operation times in practice (not worst case) and are impractical also due to the fact that each comparison of the keys consists of comparing billions of digits. In a hash table, we have to compare the digits only when the hash functions are the same.

(b) Upon registering on a social media platform, each user gets an automatic generated ID number. Your job is to maintain the data of all users that deleted their accounts. Since the data of deleted user accounts is (officially) kept only for legal reasons, the time to access and read such data does not matter. However the insertion time is very important as the platform wants to minimize the total worst-case computational time (and thus the actual cost) spent on all the users that left.

Solution: Correct answer: Unsorted (dynamic) array (or queue, or stack, or an unsorted variant of the linked list where we always insert at the end).
Explanation: The best possible total time to add n users is $\Theta(n)$. This time is achieved with the unsorted array, queue, stack and linked list because they need only $O(1)$ (amortized) time for a single add operation. In other data structures (sorted array, heap, AVL tree, hash table), the time for a single add operation is at least $\Omega(\log n)$ in the worst (and amortized) case, making the total worst case time at least $\Theta(n \log n)$. Note that hash tables achieve $O(1)$ expected time per operation—which is sufficient in practice—but for a carefully crafted sequence of keys, a single operation can take $\Theta(n)$ in the worst case, resulting in a total worst-case time of $\Theta(n^2)$.

Task 5.5: *Heap and BST*

(a) The heap introduced in the lecture is actually called a *max*-heap, as its name-giving operations `FindMax()` and `RemoveMax()` are with respect to the largest element value inside the heap.

Design a *min*-heap using a max-heap as a black box! That is, implement a data structure that has only the following operations and that internally uses a max-heap:

- `Init()`: Initializes an empty min-heap.
- `Add(`v`)`: Adds the value v to the heap
- `FindMin()`: Returns the minimum value in the heap.
- `RemoveMin()`: Deletes an element with the minimum value in the heap and returns this value.

The running times should be asymptotically the same as the corresponding operations in the max-heap. You may assume that the elements are numbers and that the array underlying the max-heap is dynamic.

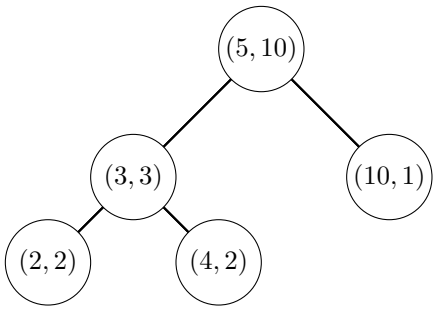
Solution: Idea: Just multiply incoming numbers by -1 and again by -1 before returning them.

```
minHeap.Init()
    maxHeap.Init()

minHeap.Add(v)
    maxHeap.Add(-v)

minHeap.findMin()
    return −maxHeap.findMax()

minHeap.removeMin()
    return −maxHeap.removeMax()
```

(b) Draw a binary search tree that additionally satisfies the heap property with respect to the values containing the following key-value pairs:

$$(3,3),(2,2),(4,2),(10,1),(5,10)$$

Solution: Start with the root of the tree which has to contain the pair with maximum value (heap property). By the binary search tree invariant, you know which nodes have to lie to the left and which ones to the right to the root (compare the keys). Recursively apply the two rules. The resulting (unique) tree:



(c) Design an algorithm `Successor`($root$, k) in pseudocode that returns the smallest key that is larger than k among the keys of the binary search tree rooted at $root$. You can assume that k and its successor exist in the tree.

Solution: The algorithm works in two steps. First we find a node (possibly the root) with the key k, or—if it does not exist—the empty child where an element with key k would be inserted. Secondly, we look for the successor of that node—see `SuccessorOfNode(`$node$`)`. There are two cases. If the node has a right child, then we look for the smallest key in the subtree of its right child, hence, we call `findMin..()` for that subtree. If the node does not have a right child, then we look for the first grandparent with larger key; hence, we do a `findMin..()` in the tree above. We also consider the case that there is no key larger than k.

```
findMinKeyNode(node)
    while node.left ≠ NONE do
        node = node.left
    return node
findMinKeyNodeAbove(node)
    while node.parent ≠ NONE and node.parent.left ≠ node do
        node = node.parent
    return node.parent
SuccessorOfNode(node)
    if node.right == NONE then
        return findMinKeyNodeAbove(node)
    return findMinKeyNode(node.right)
```

```
Successor(root, k)
    node = root
    par = NONE
    while node ≠ NONE and node.key ≠ k do
        par = node
        if k < node.key then
            node = node.left
        else
            node = node.right
    // The first if handles the (optional) case that the key k does not
       exist in the tree.
    if node == NONE then
        if par == NONE then
            return FAIL
        else if k < par.key then
            return par.key
        node = par
    node = SuccessorOfNode(node)
    if node == NONE then
        return FAIL
    return node.key
```

(d) Design an algorithm `SortedOutput`(*root*) in pseudocode that prints all the keys of the binary search tree rooted at *root* in sorted order. The running time should be linear in the number of elements. If you visit a node *node*, you can print its key by calling `print`(*node.key*).

Solution: A straighforward idea is to first find the node with the minimum key and print its key. Then, repeatedly, to look for the next successor using the auxiliary operations from the preceding task. The time of the solutions is linear in n as we walk along each edge at most twice: once down and once up.

```
SortedOutput(root)
    node = findMinKeyNode(root)
    while node ≠ NONE do
        print(node.key)
        node = SuccessorOfNode(node)
```

But there is a much simpler recursive solution:

```
SortedOutput(root)
    if root ≠ NONE then
        SortedOutput(root.left)
        print(root.key)
        SortedOutput(root.right)
```

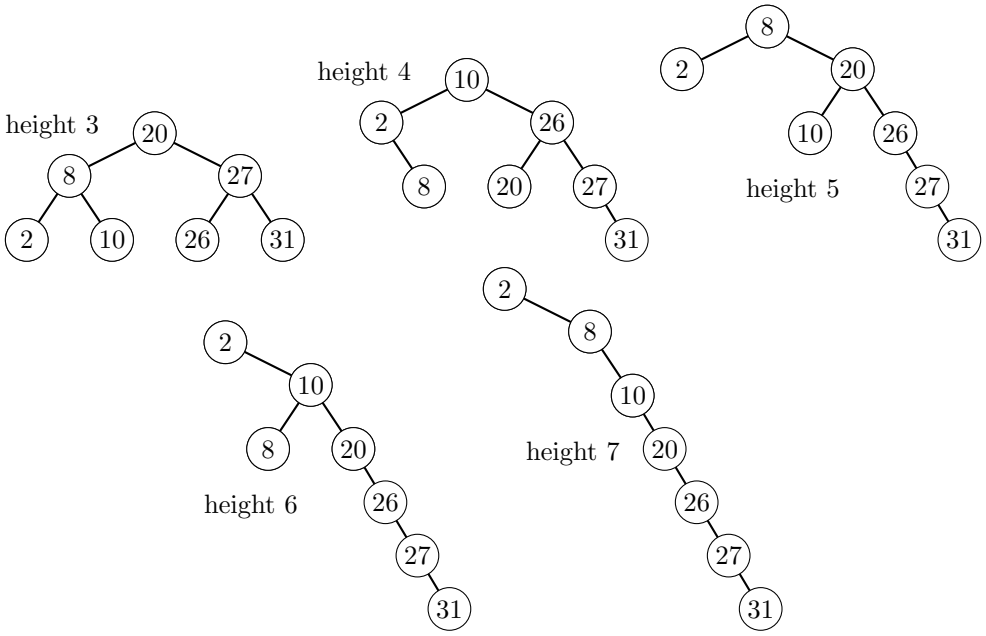
The time is linear in n, since we call the function at most once for every node and at most $2n$ for the value NONE (as every node has at most two NONE children).

(e) Draw binary search trees of heights 3, 4, 5, 6 and 7 each containing the following keys:

$$2, 8, 10, 20, 26, 27, 31 \ .$$

Recall that the height of a tree is the number of nodes on a longest path from the root.

Solution: For the heights 4 to 7 there is more than one solution.



Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 5.6: *Interval Trees*

Design a data structure that maintains a dynamic set of intervals and allows the following operations, each running in $O(\log n)$ time, where n is the current number of intervals stored in the data structure.

1. `Search`$([a, b])$: returns true if your data structure contains the interval $[a, b]$, otherwise returns false.
2. `Insert`$([a, b])$: adds the interval $[a, b]$ to your data structure (if it does not exist yet).
3. `Delete`$([a, b])$: removes the interval $[a, b]$ from your data structure (if it exists).
4. `Intersects`$([a, b])$: checks whether your data structure contains at least one interval that has a non-empty intersection with the interval $[a, b]$.

Solution: Use an AVL tree where each interval $[a, b]$ is stored in a node with the key (a, b). The keys are compared lexicographically to each other, that is, first by the left endpoint of the intervals, and then, if they are the same, by the second endpoint.

This already enables us to perform the first three operations (`Search`, `Insert` and `Delete`) in $O(\log n)$ time.

To be able to do also the `Intersects` operation efficiently, we augment the tree by storing in each node v the right-most endpoint of any interval stored in the subtree rooted at v. We store it in a new attribute called **max_end** . One can easily show that during the other operations this attribute can be updated efficiently as it can change only in nodes along the path from the root to the respective node (that we searched, inserted or deleted). Hence, the update is within the running time of the other operations.

Our algorithm for `Intersects`($[a, b]$ uses

1. the observation: $[a, b]$ intersects an interval $[start, end]$ if andd only if $start \leq b$ and $end \geq a$.
2. the invariant: if there is some interval intersecting $[a, b]$, then it is contained in the current subtree.

$v = root$
while *still no answer* **and** $v \neq NONE$ **and** v.**max_end** $\geq a$ **do**
  if $[a, b]$ *intersects the interval of* v **then**
    return True
  else if $[a, b]$ *lies completely to the left of the interval of* v **then**
    $v = v$.**left_son**
  else `// [a, b] lies completely to the right of the interval of` v
    if v.**left_son**.**max_end** $\geq a$ **then**
      `// [a, b] intersects an interval that start to the left of` v`.`
      return True
    else
      $v = v$.**right_son**
return False

Algorithms for Data Science
Cheatsheet for Exercises 6

Notations in Graph Theory:

- n is the number of vertices.
- m is the number of edges.
- If you draw a graph, you are allowed to draw edges also as curves (if not otherwise noted).
- *node* is a different word for *vertex*.

- A *path* is a sequence of consecutive edges, where the next edge starts where the last edge ends.
  - The *length* of a path is the number of its edges.
  - The *cost* (*weight*) of a path is the total weight of its edges.
  - A path is *simple* if it has no repeating vertices.
- A *cycle* is a simple path that ends where it starts.
- A *tour* is a (not necessarily simple) path that ends where it starts and that visits all the vertices of the graph.
- A graph is *connected* if one can reach any vertex from any other one by a path.
- A *tree* is a connected undirected graph without cycles.

- A *subgraph* is any graph that contains a subset of the vertices and edges of the original graph. For example, every graph is a subgraph of itself.
- A *spanning subgraph* of a connected graph is any connected subgraph containing all the vertices.
- A spanning subgraph is *minimal* if its total edge weight is minimal.

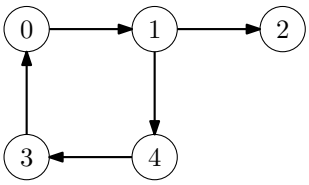
If not otherwise noted, we assume graphs to be undirected and unweighted and to have neither parallel edges nor self-loops (a self-loop is an edge that starts and ends in the same vertex).

Algorithms for Data Science
Exercise 6
(with Solutions)

See the cheatsheet for valuable remarks!

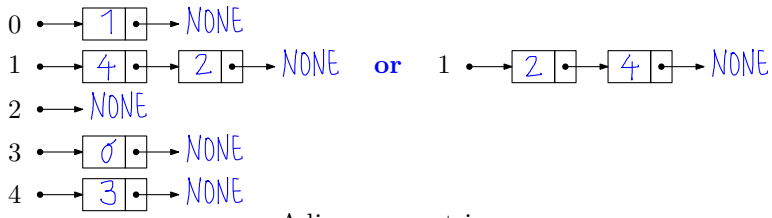
Task 6.1: *(Simple Tasks) Graphs, Stack and BFS*

(a) Given the following directed graph, write the corresponding adjacency matrix and adjacency lists.



Solution:

Adjacency lists:



Adjacency matrix:

	0	1	2	3	4
0	0	**1**	0	0	0
1	0	0	**1**	0	**1**
2	0	0	0	0	0
3	**1**	0	0	0	0
4	0	0	0	**1**	0

(b) Write the outputs of the following sequence of operations when the underlying data structure is an empty Stack.

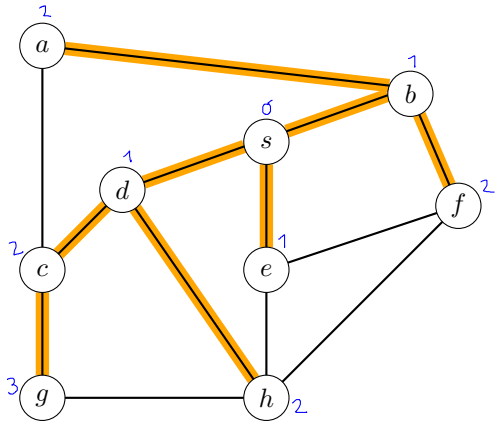
`Add(0)`, `Add(4)`, `FindNewest()`, `Add(3)`, `ExtractNewest()`, `Add(2)`, `ExtractNewest()`, `Add(1)`, `ExtractNewest()`, `ExtractNewest()`, `FindNewest()`, `Add(2)`, `ExtractNewest()`, `ExtractNewest()`

Solution: Only the operations `FindNewest()` and `ExtractNewest()` make outputs. We obtain:

$$4, 3, 2, 1, 4, 0, 2, 0$$

(c) Breadth-first Search (BFS) is run on the following graph with starting node s. Order the vertices by their order of being extracted from the queue and compute their distances and parents. (If a vertex has more neighbors, we consider them in alphabetical order.)

```
dist[s] = 0
parent[s] =NONE
q=new queue
q.add(s)
while q not empty do
    v = q.extractOldest()
    for every edge (v, w) do
        if dist[w] not computed yet
         then
            dist[w] = dist[v] + 1
            parent[w] = v
            q.add(w)
```

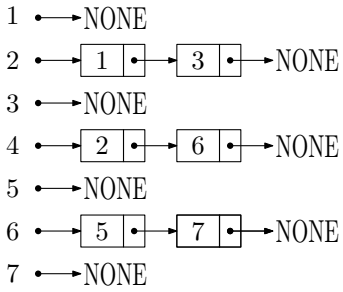


Solution:

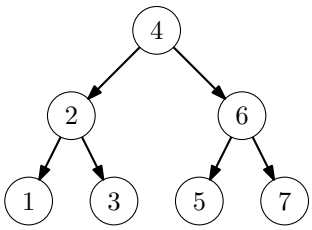
	s	a	b	c	d	e	f	g	h
distance	0	2	1	2	1	1	2	3	2
parent	NONE	b	s	d	s	s	b	c	d
removal order	1	5	2	7	3	4	6	9	8

Task 6.2: *Drawing Graphs*

(a) Draw a directed graph without edge crossings corresponding to the following adjacency lists.



Solution:

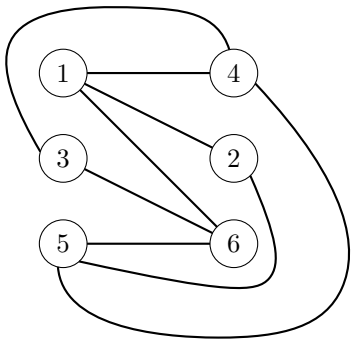


Coincidentally or not, the graph looks like a complete binary search tree.

(b) Draw a (undirected) graph without edge crossings corresponding to the following adjacency matrix.

	1	2	3	4	5	6
1	0	1	0	1	0	1
2	1	0	0	0	1	0
3	0	0	0	1	0	1
4	1	0	1	0	1	0
5	0	1	0	1	0	1
6	1	0	1	0	1	0

Solution:



A side note: The graph is a *bipartite* graph and it nearly corresponds to the *complete* bipartite graph $K_{3,3}$. In a bipartite graph, the vertices belong to two groups and there are only edges between the two groups. In our case, the vertices with odd and even numbers form two such groups. A complete bipartite graph contains all possible edges between the two groups. In our case, we miss the edge $(2, 3)$ to be complete. And this is no coincidence as the complete bipartite graph with three vertices in each group, called $K_{3,3}$, cannot be drawn without crossings. You do not believe? Then try it! Or rather try to prove that it is not possible.

The two smallest graphs that cannot be drawn without crossings are $K_{3,3}$ and the graph K_5 which is defined as the graph that has five vertices and contains every possible edge. A famous result in graph theory states that a graph is planer (i.e., it can be drawn without edge crossings) if only if it does neither contain $K_{3,3}$ nor K_5 as a so-called *minor* (a subgraph obtained via edge deletions and so-called edge contractions). Interestingly, one can decide in linear time whether a given graph is planar and even compute a planar (i.e., crossing-free) drawing in the same time!

Task 6.3: *Waiting in the Queue!*

Write the outputs of the following sequence of operations when the underlying data structure is an empty Queue.

`Add(0)`, `Add(4)`, `FindOldest()`, `Add(3)`, `ExtractOldest()`, `Add(2)`, `ExtractOldest()`, `Add(1)`, `ExtractOldest()`, `ExtractOldest()`, `FindOldest()`, `Add(2)`, `ExtractOldest()`, `ExtractOldest()`

Solution: The output writes the elements in the order of how they have been added, thus:

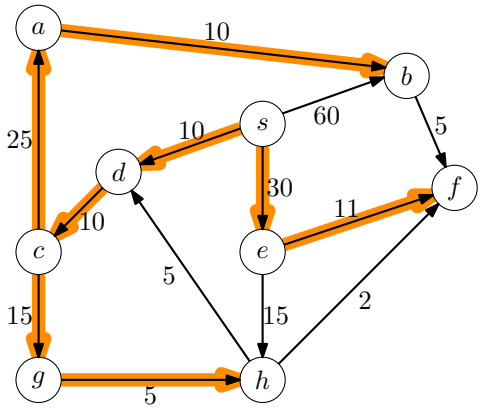
$$0, 0, 4, 3, 2, 1, 1, 2$$

Task 6.4: *Dijkstra*

Dijkstra's algorithm (see below) is run on the following graph with starting node s. Compute the distances and parents of all nodes. Also mark all edges that belong to cheapest paths from s.

Hint: While solving, write the current distances next to the vertices and mark those vertices that have already been removed from the queue.

```
dist[0..n − 1] filled with INF
parent[0..n − 1] filled with NONE
dist[s] = 0
q=new priority queue
q.add(s,0)
while q not empty do
    v = q.extractMax()
    if v removed for the first time
     then
        for every edge (v, w) do
            if dist[w] > dist[v] + c_{v,w}
             then
                dist[w] = dist[v]+c_{v,w}
                parent[w] = v
                q.add(w, -dist[w])
```



Solution:

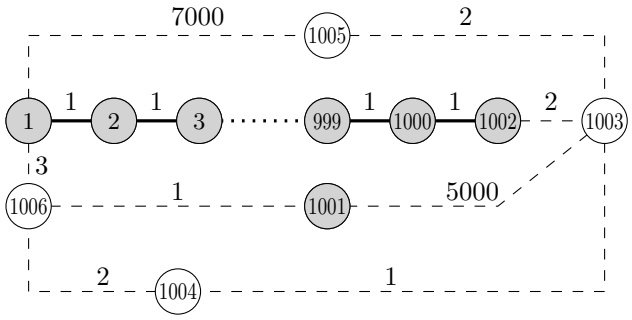
	s	a	b	c	d	e	f	g	h
distance	0	45	55	20	10	30	41	35	40
parent	NONE	c	a	d	s	s	e	c	g

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 6.5: *Incomplete Floyd-Warshall*

Floyd-Warshall's algorithm (see below) is executed on the following huge undirected graph where the vertices are numbered from 1 to n=1006 and the edge weights are given as below (the edges on the path from 3 to 999 have weight 1).



Unfortunately, the algorithm as written below has an **error** because it stops the main for loop (line 9) already after 1002 iterations.

```
1  δ[1..1006][1..1006] = new 2D array filled with +∞
2  for i = 1 to 1006 do
3      δ[i][i] = 0
4  foreach edge (v, w, c) do
       // v and w are the endpoints and c is the weight of the edge
5      δ[v][w] = c
6      δ[w][v] = c
7  for k = 1 to 1002 do
8      foreach pair (v, w) do
9          δ[v][w] = min(δ[v][w],    δ[v][k] + δ[k][w])
10 return δ
```

Write the values of the distance table for the following cells as computed by the **erroneous** algorithm above. That is, we still assume that the loop in line 7 iterates **only 1002 times**!

$\delta[1][1002] =$	**1000**	$\delta[1][1001] =$	$+\infty$	$\delta[1][1005] =$	**7000**
$\delta[2][4] =$	**2**	$\delta[1003][1006] =$	**1005**	$\delta[1001][1005] =$	$+\infty$
$\delta[1003][1004] =$	**1**	$\delta[1001][1004] =$	$+\infty$		

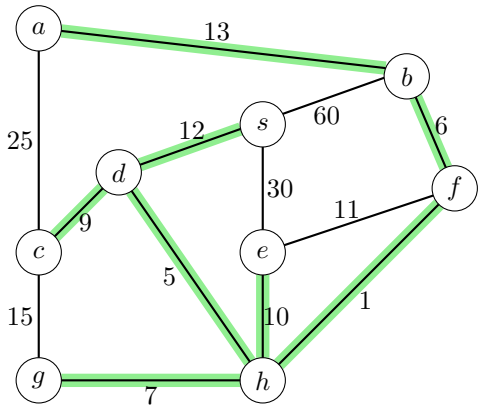
Solution: According to the invariant that we discussed in the lecture, after 1002 iterations ($k = 1002$), the Floyd-Warshall algorithm has computed for every pair of vertices the cost of a cheapest path that uses only vertices 1 to 1002 (shaded gray in the figure). In other words, for each pair of

vertices that we are asked for, we look for a cheapest path that is either a direct edge or that uses internally only gray vertices. If there is no such path, we output infinity!

Task 6.6: *Minimum Spanning Tree*

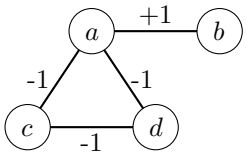
(a) Draw a minimum spanning tree in the following graph.

Solution: Take the cheapest edge that does not produce a cycle, and repeat.



(b) A minimal spanning subgraph is always a tree if the edge weights are positive. Draw a weighted graph that has positive and negative edge weights and whose minimal spanning subgraph is not a tree.

Solution: For instance, in the following graph, a minimal spanning subgraph has to take all the edges and thus contains a cycle.



Task 6.7: *Salt and Pepper*

For cooking one needs only salt and pepper. Among a group of n people, give every person either a salt shaker or a pepper shaker such that when any two friends meet, they can cook together. If such a distribution is not possible, say so. In total there are k pairs of friends. (It is possible that one person has several friends in the group.)

Model the problem as a graph problem and use (or modify) an appropriate graph algorithm from the lecture to solve it. Your solution should work in time $O(n + k)$.

(a) First, assume that your graph is connected!

(b) Next, give an algorithm that also works if the graph is not connected! (*Advanced task*: is your running time really $O(n + k)$?)

Solution: Solution sketch (for both cases: that the graph is connected and not necessarily connected):

- vertices = people
- edges = pairs of friends
- Color problem: Say whether it is possible to color the vertices in white and black such that no two neighboring vertices have the same color. (If it is possible, return such a coloring.)
- Observation 1: If there is a solution, then we can swap black and white and it is still a solution.
- Observation 2: If a vertex has one color, then all its neighbors must have the other color.
- Observation 3 (a consequence of Observation 2): If a vertex s is, say, black, then all vertices with odd distance to s must be colored white and all vertices with even distance must be colored black (provided that a solution exists).
- Algorithm 1: For each vertex, if it is not colored yet, run BFS from it and color all visited vertices according to Observation 3 (choosing any color for the start vertex; see Observation 1). At the end, go through every edge and check whether its endpoints have different colors. Total time as required.
- Note: If the graph is connected, the foreach loop of Algorithm 1 will run only once (we take an arbitrary vertex as our start vertex and then reach and color all the other vertices).
- Modification: While running BFS, we can already check whether the coloring is valid.

Running time analysis:

- BFS takes $O(n+k)$ time. (Recall that k is now the number of edges.)
- Checking at the end whether the endpoints of each edge have different colors takes $O(k)$ time.
- If the graph is connected (we are allowed to assume it), we run BFS only once and the total time is thus $O(n+k)$.
- If the graph is not connected, we run BFS several times. *(This is the solution of an* advanced task *and I don't expect you to come up with such an analysis!)* The fact that we run BFS several times might lead to higher total running time than $O(n+k)$. For instance, if the graph contains no edges at all, then we run BFS n times. This would lead to a running time of at least $\Omega(n^2)$ as, each time, BFS has to initialize an empty distance table of length n which already takes $\Theta(n)$ time. However, we can modify BFS such that, at each subsequent call, it uses the same distance table from the last call (without changing the values). If BFS does not have to initialize the distance table, then it running time drops to $O(n_s + k_s)$ where n_s and k_s are the number of vertices and edges, respectively, considered by BFS which are exactly the vertices and edges reachable form the starting vertex s. Hence, if S is the set of all vertices from which we run BFS in our algorithm (their number equals the number of connected components in the graph and, in particular, we have $\sum_s n_s = n$ and $\sum_s k_s = k$), then the total running time of our algorithm is (which consists of the time $O(n)$ to initialize the distance table, then the time to run BFS from every vertex from S, as well as to check all edges at the end):

$$O(n) + \sum_s O(n_s + k_s) + O(k) = O(n) + O(n+k) + O(k) = O(n+k)$$

Task 6.8: *Is it true . . . ? (Graphs)*

(a) Is it true that any connected graph with $n-1$ edges has no cycles? Prove the claim or draw a counter example.

Solution: It is true.
One of many possible proofs:

Suppose for contradiction that there is a cycle. As long as there is a cycle, remove an edge from the cycle. Note that the graph still is connected afterwards but now it has less than $n-1$ edges. In other words, there are at least two more vertices than edges.

Next, as long as there is a vertex with exactly on edge, delete the vertex and the edge. By this operation, we decrease the number of vertices by one and the number of edges by one, and the remaining graph is still connected and still has at least two more vertices than edges. Note that the remaining graph has at least one vertex.

If the process stops because all remaining vertices have at least two edges, then we actually have cycle. But recall that we have already destroyed all cycles. So this situation can't happen.

Thus, the process has to stop because there are no edges left. Consider the moment when we remove the last edge in this process. As shown above, there are at least two more vertices than the number of edges. Hence, after removing the last edge with exactly one of its endpoint, we are left with two vertices and no edge. Hence, the remaining graph is disconnected which contradicts our observation that it has to be connected after each step of the process. This means that our initial assumption that the initial graph had a cycle is wrong. Hence, there are no cycles in the initial graph.

Note that connected (undirected) graphs with $n-1$ edges are exactly the graphs that we call trees.

(b) Also in this task, we consider only graphs without parallel edges. What is the maximum number of edges . . .

i. . . . in a directed graph when we allow self-loop edges?

ii. . . . in a directed graph when we do not allow self-loop edges?

iii. . . . in an (undirected) graph when we do not allow self-loop edges?

Solution: The answers are: n^2, $n(n-1)$, $n(n-1)/2$.
Explanations: (i) If we allow self loop edges, then every vertex has a directed edge to every other vertex (including itself). Hence, there are $n \cdot n = n^2$ edges.
(ii) As above, but now every vertex has only a directed edge to every other vertex (not including itself). Hence, there are $n \cdot (n-1)$ edges.
(iii) As above, but we have to divide it by 2 as above we counted every edge twice. In other words: For every undirected edge, we counted two directed edges above (one in one direction and the other one in the other one).

Task 6.9: *Floyd-Warshall vs Dijkstra*

In the lecture, we have seen that the problem of computing the cheapest path from every vertex to every other can be solved via the Floyd-Warshall algorithm in time $\Theta(n^3+m)$ or by running Dijkstra's algorithm from every vertex in total time $\Theta(mn \log m + n^2)$ (on the slides we further simplified these running times assuming that there are no parallel edges).

What are the running times of the two approaches if . . .

(a) ... m is extremely small, that is, $m = \Theta(1)$? Give the running times in dependence of only n.

Solution: If $m = O(1)$, then Floyd-Warshall runs in $\Theta(n^3)$ and Dijkstra in $\Theta(n^2)$.

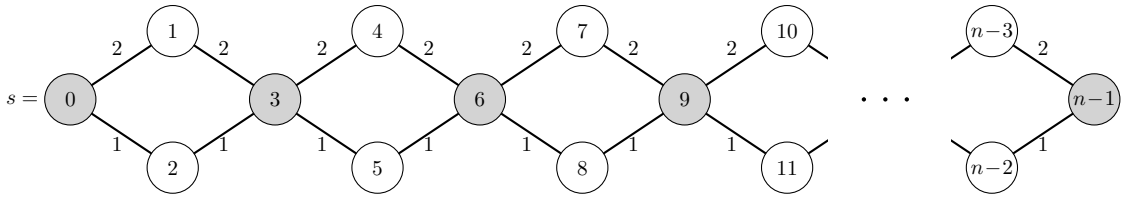
(b) ... m is extremely large, that is, $m = \Theta(n^4)$ (thus, we have parallel edges)? Give the running times in dependence of only m.

Solution: If $m = \Omega(n^4)$, then Floyd-Warshall runs in $\Theta(m)$ and Dijkstra in $\Theta(m^{1+1/4} \log m)$.

Task 6.10: *Dijkstra with a Normal Queue*

In this task, you learn why it is very important to use a priority queue in Dijkstra's algorithm.

Consider the following graph. Argue that, in the worst case, Dijkstra's algorithm will run immensenly much slower on that graph if we use a normal queue (as in the algorithm below) instead of a priority queue!



$dist[0..n-1]$ filled with INF
$dist[s] = 0$
q=new queue
q.add(s)
while q *not empty* **do**
  $v = q$.removeOldest()
  if v *removed for the first time* **then**
    for *every edge* (v, w) **do**
      if $dist[w] > dist[v] + c_{v,w}$ **then**
        $dist[w] = dist[v] + c_{v,w}$
        q.add(w)

Solution: You will notice that in the worst case, if a vertex has more than one neighbor, we consider them from top to bottom (equivalently, in increasing order of their numbers). Thus, first the neighbor with the more costly edge, then the neighbor with the cheaper edge. If you try to run the algorithm by hand, you will notice that the vertices are added to the queue in this order:

$$\mathbf{0}, 1, 2, \mathbf{3}, \mathbf{3}, 4, 5, 4, 5, \mathbf{6}, \mathbf{6}, \mathbf{6}, \mathbf{6}, \ldots$$

You see that the same vetices are added over and over again. If you focus on the gray vertices (in bold above), which are exactly the vertices whose numbers are dividible by 3, you will see that each next gray vertex is added twice as often as the preceding gray vertex. In other words, the gray vertex with number $3k$ is added 2^k times to our queue. (You can also prove this fact with induction—see the cheatsheet to the first exercise sheet.) Consequently, the last gray vertex with the number $n-1$ is added $2^{(n-1)/3} = \Theta(c^n)$ times, where c is the constant $\sqrt[3]{2}$. Hence, the last vertex is added exponentially often.

Thus, the total running time is at least exponential, which is extremely much slower than Dijkstra's almost linear running time of $O(n + m \log n)$.

Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 6.11: *Output the Cheapest Paths Using Floyd-Warshall*

(a) Extend the algorithm of Floyd-Warshall such that it outputs the actual cheapest paths in additional time $O(n)$ for each pair.

Note: Print the vertices as they appear on the path, one by one.

Solution: Modify Floyd-Warshall to also compute a 2D table *next* such that $next[u][v]$ is the sucessor of u on the cheapest path from u to v. (Initially, set $next[u][v] = v$ for every pair u, v.) Whenever you set $\delta[u][v] = \delta[u][k] + \delta[k][v]$ (which happens only if $u \neq k$), update $next[u][v] = next[u][k]$.

Once *next* is computed, you can recursively reconstruct a cheapest u-v-path as follows:

```
k = u
while k ≠ v do
    Print(k)
    k = next[k][v]
Print(k)
```

The time is obviously $O(n)$ for reconstruction. The time to compute *next* does not increase the $O(n^3)$ time of Floyd-Warshall.

(b) Now suppose that you are only given the δ table as computed by Floyd Warshall and that you don't have access to the graph. Give an $O(n^2)$-time algorithm that, given a pair of vertices u and v, reconstructs a cheapest path from u and v. Assume that the δ table already lies in the memory and that it takes $O(1)$ time to read a single entry of the table.

Note: Print the edges as they appear on the path, one by one.

Solution: Backtrack the Floyd-Warhshall algorihtm by reconstructing which decisions had been made. You can do it, for example, recursively as follows.

```
ReconstructPath(u,v)
    return AlgoRec(u, v, n)

AlgoRec(u, v, k)
    if k == 0 then
        if u ≠ v then
            Print((u, v))
    else
        if δ[u][v] == δ[u][k] + δ[k][v] then
            AlgoRec(u,k,k − 1) AlgoRec(k,v,k − 1)
        else
            AlgoRec(u,v,k − 1)
```

Or you can do it iteratively using a stack; which, by the way is a nice example of how stacks are useful to transform a recursive algorithm into an iterative one. In fact, a recursive algorithm

has always a so-called recursion stack so that the computer remembers how to go back and what to do next when coming back from a recursion.

```
s = new empty stack
s.Add((u, v, n))
while s not empty do
    (u, v, k) = s.ExtractNewest()
    if k == 0 and u ≠ v then
        Print((u, v))
    else if k > 0 then
        if δ[u][v] == δ[u][k] + δ[k][v] then
            s.Add(k,v,k − 1)
            s.Add(u,k,k − 1)
        else
            s.Add(u,v,k − 1)
```

To analyze the time, first observe that the condition $\delta[u][v] == \delta[u][k] + \delta[k][v]$ is satisfied at most once for every k, hence, it happens at most $O(n)$ times. Thus, at most $O(n)$ times, we start to consider two new vertex pairs. Each vertex pair is considered not more than n times as k decreases by 1 each time. Hence, we have at most $O(n^2)$ triples that we consider (and each triple is considered at most once). This discussion implies the running time.